

西北工业大学
NORTHWESTERN POLYTECHNICAL UNIVERSITY

软件需求工程

Software Requirements Engineering

西北工业大学 软件学院



第二章

需求获取



第二章 需求获取



本章主要内容：

- 2.1 需求获取概述
- 2.2 需求分析人员
- 2.3 建立项目视图与范围
- 2.4 寻找客户的需求
- 2.5 基于用例的方法
- 2.6 软件的质量属性
- 2.7 需求获取小结

2.5 基于用例的方法

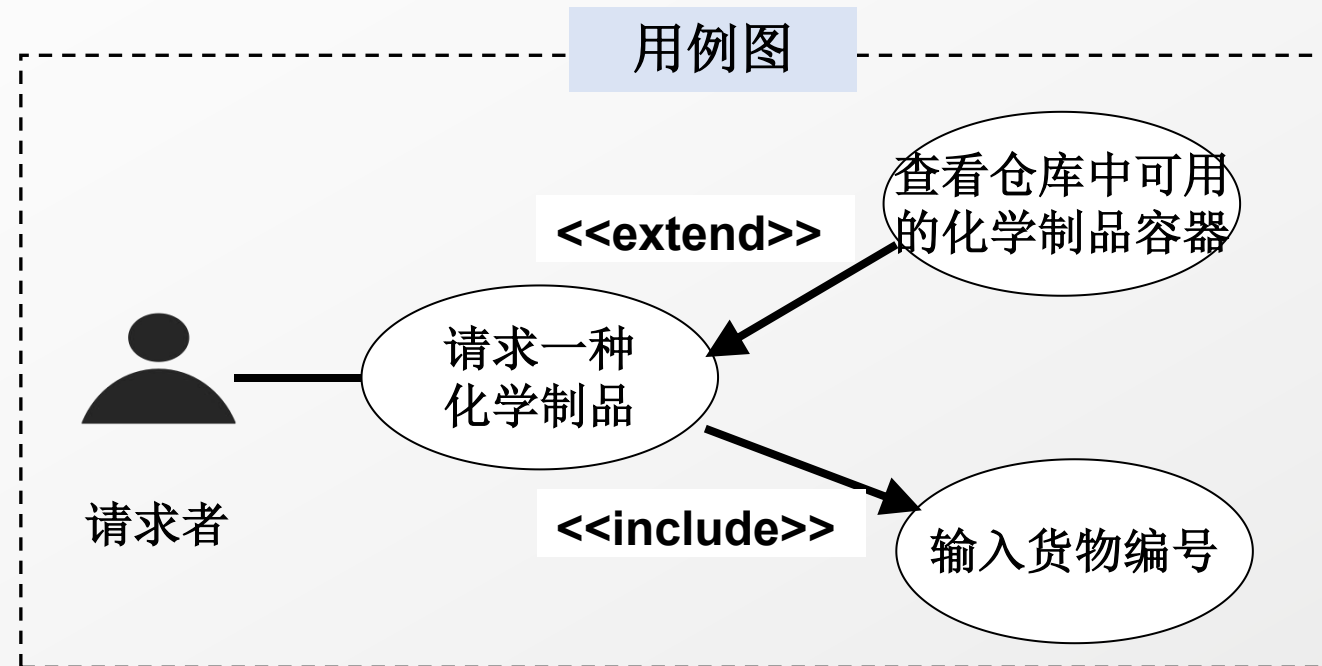
2.5.1 用例和用例说明

(1) 基本概念

需求获取是询问“用户要利用系统做什么？”

还是询问“用户希望系统为他们做什么？”

1992年，Ivar Jacobson等人系统地阐述了用例(use—case)的方法进行需求获取和建模。



2.5 基于用例的方法

2.5.1 用例和用例说明

(1) 基本概念

1) **用例**：一个用例描述了系统和一个外部“执行”（actor）的交互顺序，这体现执行者完成一项任务并给某人带来益处。

2) **执行者**：是指一个人、或另一个软件应用、或一个硬件、或其它一些与系统交互以实现某些目标的实体。执行者可以映像到一个或多个可以操作的用户类的角色。

例如：“化学制品跟踪系统”中的“请求一种化学制品”用例包括一个名为“请求者”的执行者。在“化学制品跟踪系统”中设有一个客户类叫请求者，药剂师和化学制品仓库管理人员均可充当这一角色。（泛化！Generalization）

Include：包含

Extend：扩展（Module 10 Structure the use-case model）



2.5 基于用例的方法

2.5.1 用例和用例说明

(2) 用例的特点

用例为表达用户需求提供了一种方法，而这一方法必须与**系统的业务需求**相一致。**分析者和用户必须检查每一个用例**，在把它们纳入需求之前决定其是否在**项目所定义的范围**内。

基于“用例”方法进行需求获取的目的在于：

1) 描述用户需要使用系统**完成的所有任务**

2) 在理论上，用例的结果集将包括**所有合理的系统功能**

3) 应用基于用例的方法进行需求获取，可以为用户和开发者带来**更好的效果**

2.5 基于用例的方法

2.5.1 用例和用例说明

(3) 用例的要素

1) 一个单一的用例可能包括完成某项任务的许多逻辑相关任务和交互顺序

2) 一个用例是相关的用法说明的集合

3) 一个说明(scenario)是一个用例的例子，被视为事件的普通过程(normal course)，也叫作主过程、基本过程，普通流

4) 在描述普通过程时，应列出执行者和系统之间相互交互或对话的顺序

5) 当过程中执行者和系统之间的对话结束时，执行者也达到了预期的目的。

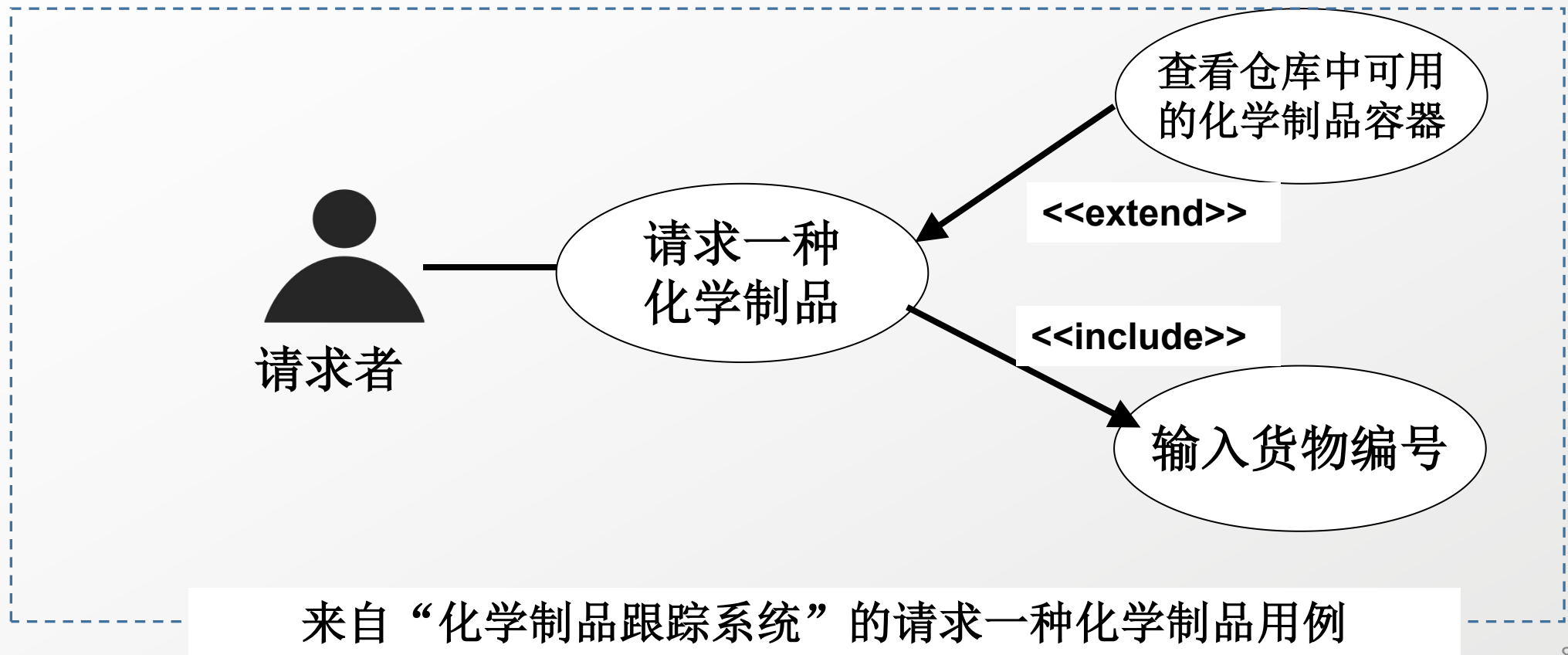
例如：“请求一种化学制品”的用例的普通过程导致一个用户请求，要求从外界供应商订购化学制品

2.5 基于用例的方法

2.5.1 用例和用例说明

(3) 用例的要素

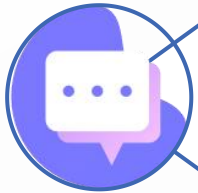
用例图：由参与者（Actor）、用例（Use Case）以及它们之间的关系构成的用于描述系统功能的动态视图



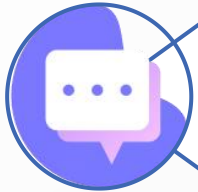
2.5 基于用例的方法

2.5.1 用例和用例说明

(4) 用例的说明



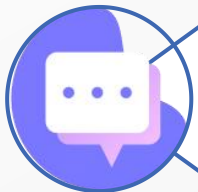
1) “请求化学制品”的用例图运用了统一建模语言(UML)概念



2) “查看仓库中可用的化学制品容器”和“输入货物编号”描述了两种可能的用例《extend》和《include》的关系



3) 用例图提供了一个高级别的用户需求的可视化表示



4) **可选过程(alternative course):** 用例中的其它说明。可选过程也可促进任务的成功完成,但它们代表了任务的细节或用于完成任务的途径的变化部分

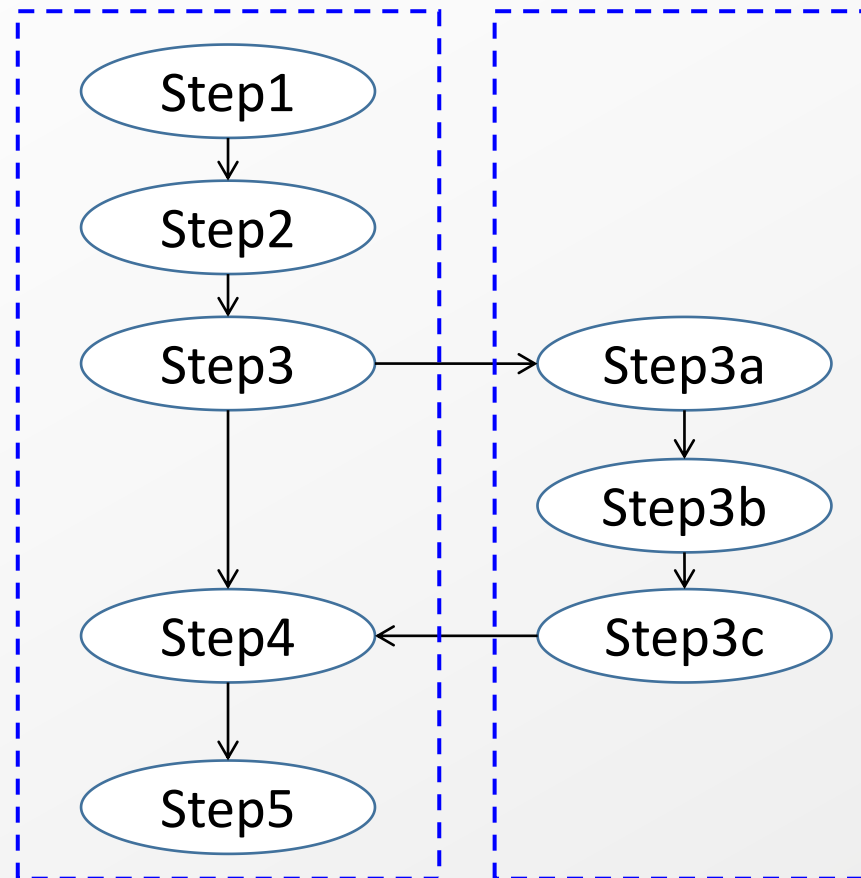
2.5 基于用例的方法

2.5.1 用例和用例说明 (4) 用例的说明

可选过程中的一些步骤与在普通过程中的步骤相同，但是需要一些唯一的动作来完成可选路径

普通过程

可选过程



“请求一种化学制品”的用例中，“向化学制品仓库请求一种化学制品”是一个可选过程

2.5 基于用例的方法

2.5.1 用例和用例说明

(4) 用例的说明

5) 例外：引起任务不能顺序完成的情况称为例外(exception)，在某些时候它可以视为可选过程。

在定义用例时，描述例外路径是很重要的，因为它们描述了在特定条件下用户对系统如何工作的看法

“请求一种化学制品”用例中的一个例外是不存在业务上可用的化学制品



若未将例外记录在文档上，则开发者可能在设计和构造阶段忽视这些可能性



当系统遇到一个例外条件时，就会发生系统崩溃

2.5 基于用例的方法

2.5.2 确定用例并编写用例文档

(1) 用例确定方法

可以使用多种方法来确定用例：

- 1) 首先明确执行者和他们的角色，然后确定业务过程，在这一过程中每一个参与者都在为确定用例而努力。
- 2) 确定系统所能反映的外部事件，然后把这些事件与参与的执行者和特定的用例联系起来。
- 3) 以特定的说明形式表达业务过程或日常行为，从这些说明中获得用例，并确定参与到用例中的执行者。
- 4) 有可能从现在的功能需求说明中获得用例。如果有些需求与用例不一致，就应考虑是否真的需要它们。

2.5 基于用例的方法

2.5.2 确定用例并编写用例文档

(1) 用例确定方法

简化的“请求一种化学制品”用例的模板

参与者将第一个从用例中获益的人定为操作者



定义任何满足用例运行的先决条件及在完成用例后描述系统的后续条件



预先估计的使用频度为**并行使用**和**性能需求**提供了一个早期提示



分析者询问参与者想**怎样与系统交互**来完成任务

2.5 基于用例的方法

2.5.2 确定用例并编写用例文档

(2) 用例描述举例

用例标识号	UC—5		
用例名称	请求一种化学制品		
创建者	Tim	最后一次更新者	Janice
创建时间	2019	最后一次更新时间	10/27
执行者	请求者		
说明	请求者通过输入化学制品名称、ID或输入化学制品的结构来指定对化学制品的请求；系统提供给请求者一个来自化学制品仓库的新的或已用过的化学制品容器，或者让请求者向外界供货商订货		
前置条件	1. 用户的身份已通过认证 2. 该用户被授权允许申领化学制品 2. 具有在线的化学制品存货清单数据库		
请求结果	1. 请求被保存入“化学制品跟踪系统”中 2. 通过电子邮件把请求发往化学制品仓库或完成采购		
优先级	高		
使用频度	对于每个药剂师大约每周5次，对于化学制品仓库的每个成员每周100次		



2.5 基于用例的方法

2.5.2 确定用例并编写用例文档

(2) 用例描述举例

普通过程	5.0 从供应商那里请求一种化学制品	
	执行者行为	系统响应
	1. 输入化学制品的ID或者包 含化学制品结构的文件名	2. 验证化学制品的ID是否合法
		3. 询问请求者需要一个新的供应商订 单或一个来自化学制品仓库的容器
	4. 确定供应商(待续)或化学 制品仓库(可选过程5. 1)	5. <继续对话, 直到请求完成>
可选过程	5. 1从化学制品仓库中请求一种化学制品(5. 0. 4之后的分枝)	
	执行者行为	系统响应
	2. 可选择地查看任何容器的历史	1. 显示出化学制品仓库中现存的所要 求的化学容器的列表
	3. 选择一个特定的容器或发出供应商订单	



2.5 基于用例的方法

2.5.2 确定用例并编写用例文档

(2) 用例描述举例

例外	5. E.1化学制品在业务上不可用	
	执行者行为	系统响应
		1. 显示消息：不存在供应商
		2. 询问请求者是否要请求另一种化学制品或退出
	3. 请求另一种化学制品	4. 普通过程结束
包括	UC-12输入货物编号	
特定需求	系统必须可以按标准的编码形式输入化学制品的结构，这一标准来自对不同化学制品图形包的支持	
假设	输入的化学制品结构被认为是合法的	
注释和问题	Tim将查明：在危险列表的第一级上请求一种化学制品是否需要管理层的批准。预期时间11/4	

2.5 基于用例的方法

2.5.2 确定用例并编写用例文档

(3) 例外的确定



如何确定
例外？

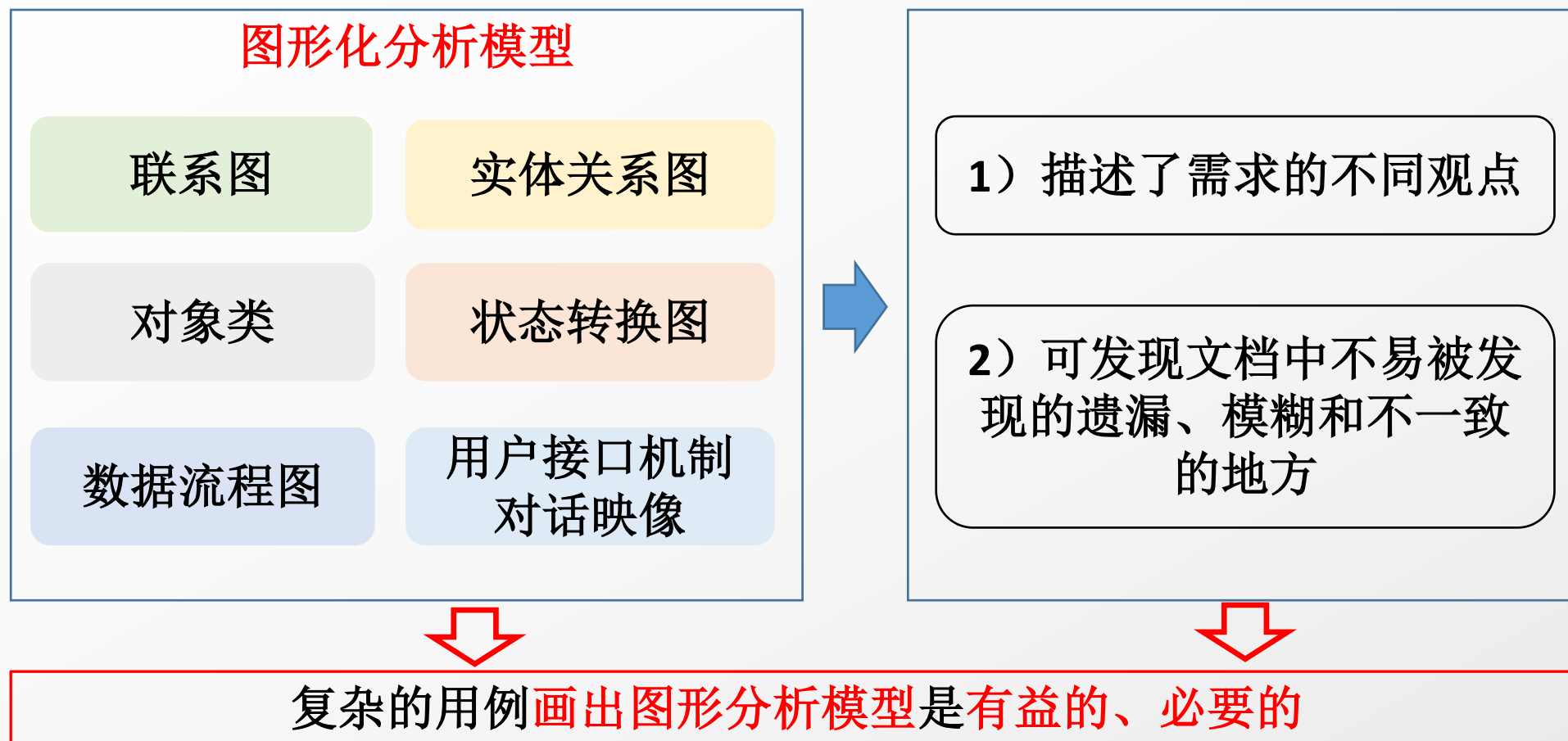
需求获取小组为可选的过程确定出相似的对话并确定例外情况。如：当分析者问一些诸如“那时，倘若数据库没有联机，将会发生什么情况？”或“当化学制品在业务上不可用时该怎么办？”

遇到这类问题时，将会发现许多例外条件。

2.5 基于用例的方法

2.5.2 确定用例并编写用例文档

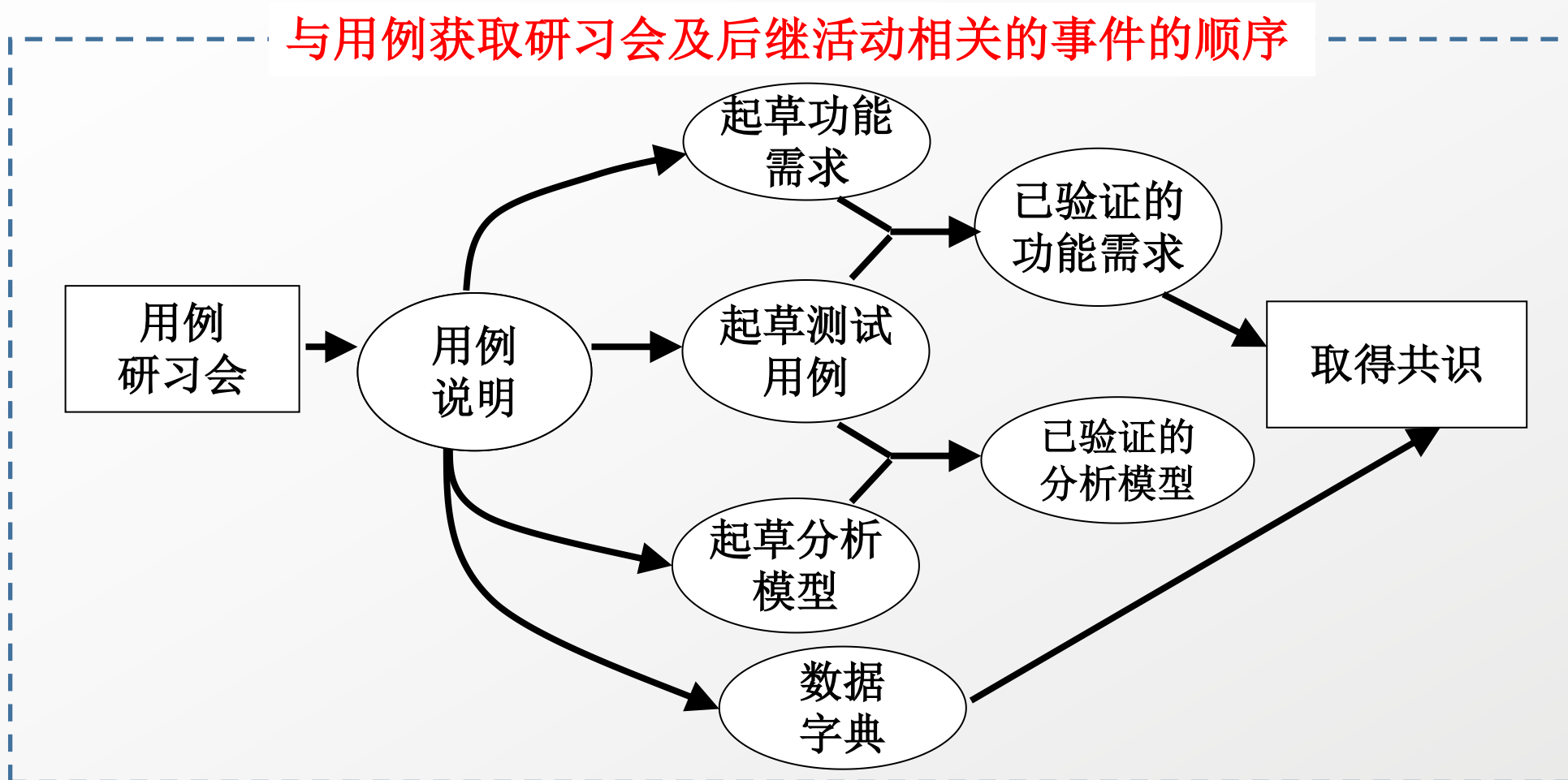
(4) 复杂用例的图形化分析



2.5 基于用例的方法

2.5.2 确定用例并编写用例文档

(4) 复杂用例的图形化分析



2.5 基于用例的方法

2.5.3 用例和功能需求

(1) 用例对建立功能需求的重要性

- 1) 用例的描述并不向开发者提供他们所要开发的功能的细节
- 2) 如果在用户需求阶段停止了需求开发，将会发现在软件的构造阶段，开发者必须询问许多问题来弥补他们的信息空白
- 3) 为了减少不确定性，你需要把每一个用例叙述成详细的功能需求
- 4) 每一个用例可引伸出多个功能需求，这将使执行者可以执行相关的任务；并且多个用例可能需要相同的功能需求

例如，如果有5个用例需要进行用户身份的验证，不必因此编写5个不同的代码块

2.5 基于用例的方法

2.5.3 用例和功能需求

(2) 用例和对应功能需求的管理

1. 仅利用用例的方法：仅将功能需求包含在每个用例描述中，还需要一个单独的补充说明来记录非功能需求，以及所有不与特定用例相关的功能需求。有些用例可能会需要相同的功能需求。

最好的方法



2. 利用用例和SRS相结合的方法：把用例说明限制在抽象的用户需求级上（即写一个简单的用例及描述），并且把从用例中获得的功能需求编入软件需求规格说明中。在这种方法中，需要在使用实例和与之相关的功能需求之间建立可跟踪性。

最好的方法



把所有用例和功能需求存入数据库或者业务需求管理工具中，这样可以定义这些跟踪性联系

2.5 基于用例的方法

2.5.3 用例和功能需求

(2) 用例和对应功能需求的管理

3.仅利用软件需求规格说明的方法：通过用例来组织软件需求规格说明，并且把使用实例和功能需求都记录在软件需求规格说明中。

- 1) 采用这种方法，你无需独立编写详细的用例文档；
- 2) 但需要确定冗余的功能需求，或者对每个功能需求仅陈述一次，且无论需求是否重复出现在其它用例中，都要参考它的原始说明。

2.5 基于用例的方法

2.5.3 用例和功能需求

(3) 用例的好处

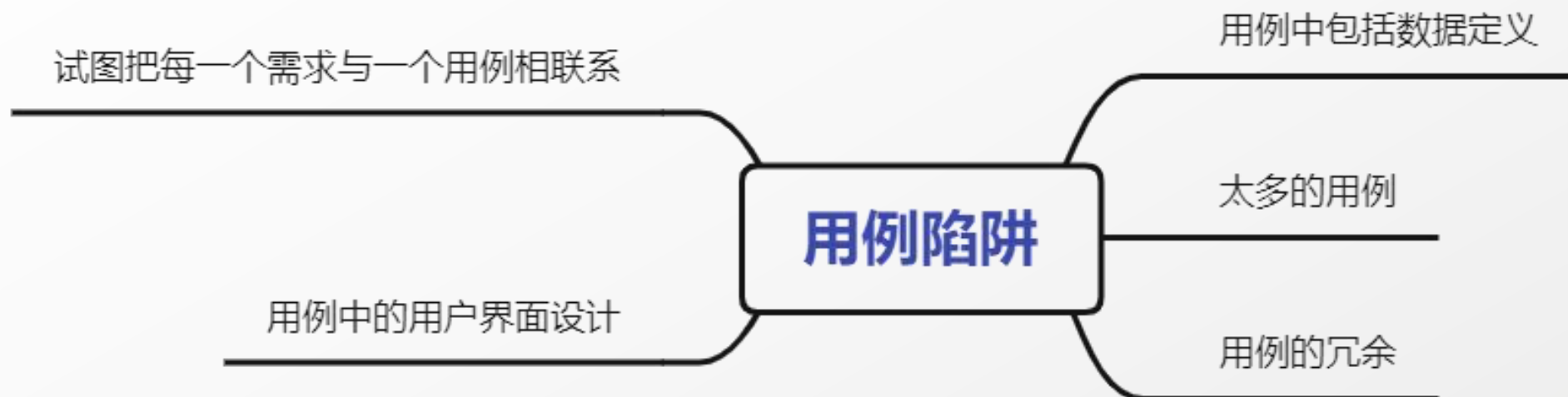
用例方法的优势：该方法是以任务为中心和以用户为中心的观点

- 1) 比起使用以功能为中心的方法，用例方法可以使用户更清楚地认识到新系统允许他们做什么
- 2) 用例有助于分析者和开发者理解用户的业务和应用领域
- 3) 认真思考执行者与系统对话的顺序，使其可以在开发过程早期发现模糊性，也有助于从用例中生成测试用例
- 4) 有了用例，所得到的功能需求会明确规定用户执行的特定任务
- 5) 开发者运用面向对象的设计方法可以把用例转化为对象模型
- 6) 跟踪功能需求、设计、编码和测试以至到它们父类的用例（即用户意见），则很容易看出整个系统中业务过程的级联变化



2.5 基于用例的方法

2.5.3 用例和功能需求 (4) 用例陷阱的避免



在用例的方法中应注意如上的陷阱

2.5 基于用例的方法

2.5.3 用例和功能需求

(4) 用例陷阱的避免

用例方法的陷阱

避免陷阱的方法

太多的用例

- 1) 不要为每一个可能的说明编写单独的用例，而是把普通过程、可选过程以及例外集成起来写入一个简单的用例
- 2) 不要把交互顺序中的每个步骤看成一个单独的用例
- 3) 每一个用例都必须描述一个单独的任务

用例的冗余

相同的函数出现在多个用例中，需要多次重写函数的实现部分。为了避免重复，可以使用“包括”关系

用例中的用户界面的设计

用例应该把**重点放在用户使用系统做什么，而不是关心屏幕上是怎么显示的**

用例中包括数据定义

有一些包括数据项和数据结构定义的用例：在这种用例中操纵这些数据项和数据结构（包括数据类型、长度、格式和合法值）

试图把每一个需求与一个用例相联系

- 1) 有一些需求与用户任务之间的交互没有特定的关系。，这时就需要一个独立的需求规格说明
- 2) 在这个规格说明中可以编写非功能需求文档、外部接口需求文档以及一些不能由用例得到的功能需求支持。

2.6 软件的质量属性

2.6.1 非功能需求

某工程项目用新的应用程序替换许多已有的主机(mainframe)应用程序。根据用户的要求，开发组设计了一个基于窗口的用户界面并定义了新的数据文件，其容量是旧文件的两倍。虽然新系统满足了技术上的规范，但并没有达到客户可接受的程度。

问题



- 1) 用户总是抱怨用户界面运行缓慢，并且新的数据文件所占用的磁盘空间太大
- 2) 用户没有陈述对新产品的一些特性的期望，这就不能在他们所提出的功能需求中体现出来
- 3) 开发者和用户没有详细地讨论新技术方法所牵涉到可能的性能，从而导致了用户期望与产品实际性能之间的期望差异
- 4) 比起仅仅满足客户所要求的功能，软件的成功似乎更为重要

2.6 软件的质量属性

2.6.1 非功能需求

某工程项目用新的应用程序替换许多已有的主机(mainframe)应用程序。根据用户的要求，开发组设计了一个基于窗口的用户界面并定义了新的数据文件，其容量是旧文件的两倍。虽然新系统满足了技术上的规范，但并没有达到客户可接受的程度。

问题



- 5) 用户总是强调确定他们的功能、行为或需求，且用户对产品如何良好地运转抱有许多期望：产品的易用程度、速度、可靠性、应对异常的能力等软件质量属性（**quality attribute**）或质量因素（**quality factor**）
- 6) 软件质量属性是系统非功能(也叫非行为)需求的一部分
- 7) 质量属性很难定义，且他们经常造成开发者设计的产品和客户满意的产品之间的差异
- 8) 现实情况中，在决定系统的成功或失败的因素中，**满足非功能需求往往比满足功能需求更为重要**



2.6 软件的质量属性

2.6.1 非功能需求

某工程项目用新的应用程序替换许多已有的主机(mainframe)应用程序。根据用户的要求，开发组设计了一个基于窗口的用户界面并定义了新的数据文件，其容量是旧文件的两倍。虽然新系统满足了技术上的规范，但并没有达到客户可接受的程度。

问题



9) 优秀的软件产品反映了这些竞争性质量特性的优化平衡

10) 如果在需求的获取阶段不去探索客户对质量的期望，那么产品更多的可能是客户失望和开发者沮丧

11) 尽管在需求获取阶段客户所提出的信息中包含提供了一些关于重要质量特性的线索，但客户通常不能主动提出他们的非功能期望。如：用户说软件必须“健壮”、“可靠”或“高效”时，这是很技巧地指出他们所需要的东西

12) 质量必须由客户及构造测试和维护软件的人员来定义。探索用户隐含期望的问题可以导致对质量目标的描述，并且制定可以帮助开发者创建完美产品的标准



2.6 软件的质量属性

2.6.2 软件质量属性

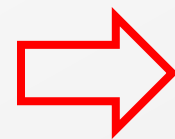
(1) 软件质量属性基本内容

虽然有许多产品特性可以称为**质量属性** (Quality Attribute)，那些特性对项目成功至关重要的因素，可通过选择软件工程方法来达到特定的质量目标：

根据不同的设计对质量属性分类

把在运行时可识别的特性与那些不可识别的特性区分开

把对用户很重要的可见特性与对开发者和维护者很重要的不可见特性区分开



那些对开发者具有重要意义的属性：

- 1) 使产品易于更改、验证
- 2) 易于移植到新的平台上，从而可以间接地满足客户的需要。

2.6 软件的质量属性

2.6.2 软件质量属性

(1) 软件质量属性基本内容

McCa11提出了表明软件质量的11个质量特性：正确性、可靠性、效率、完整性、易用性、可维护性、测试性、灵活性、可移植性、可重用性、互操作性。

质量特性	含义	可回答的问题
正确性 (Correctness)	程序能够满足规格说明和完成用户业务目标的程度	它做了该做的事吗？
可靠性 (Reliability)	程序能够按要求的精度实现其预期功能的程度	它总能准确地工作吗？
效率 (Efficiency)	程序实现其功能所需要的计算机资源量	它能在硬件上尽力工作吗？
完整性 (Integrity)	软件或数据不受未授权人控制的程度	它是安全吗？
易用性 (Usability)	学习、操作程序、为其准备输入数据、解释其输出的工作量	它易用吗？



2.6 软件的质量属性

2.6.2 软件质量属性

(1) 软件质量属性基本内容

质量特性	含 意	可回答的问题
可维护性 (Maintainability)	对运行的程序找到错误并排除错误的工作量	它可调整吗？
测试性 (Testability)	为保证程序执行其规定的功能所需的测试工作量	它可测试吗？
灵活性 (Flexibility)	修改运行的程序所需的工作量	它可修改吗？
可移植性 (Portability)	将程序从一种硬件配置和/或环境转移到另一硬件配置和/或环境所需的工作量	可以在另一台机器上用它吗？
可重用性 (Reusability)	程序可被用于与其实实现功能相关的其他应用问题的程度	可以重复使用它的某些部分吗？
互操作性 (Interoperability)	一系统与另一系统协同运行所需的工作量	它能够与另外系统联接吗？

2.6 软件的质量属性

2.6.2 软件质量属性

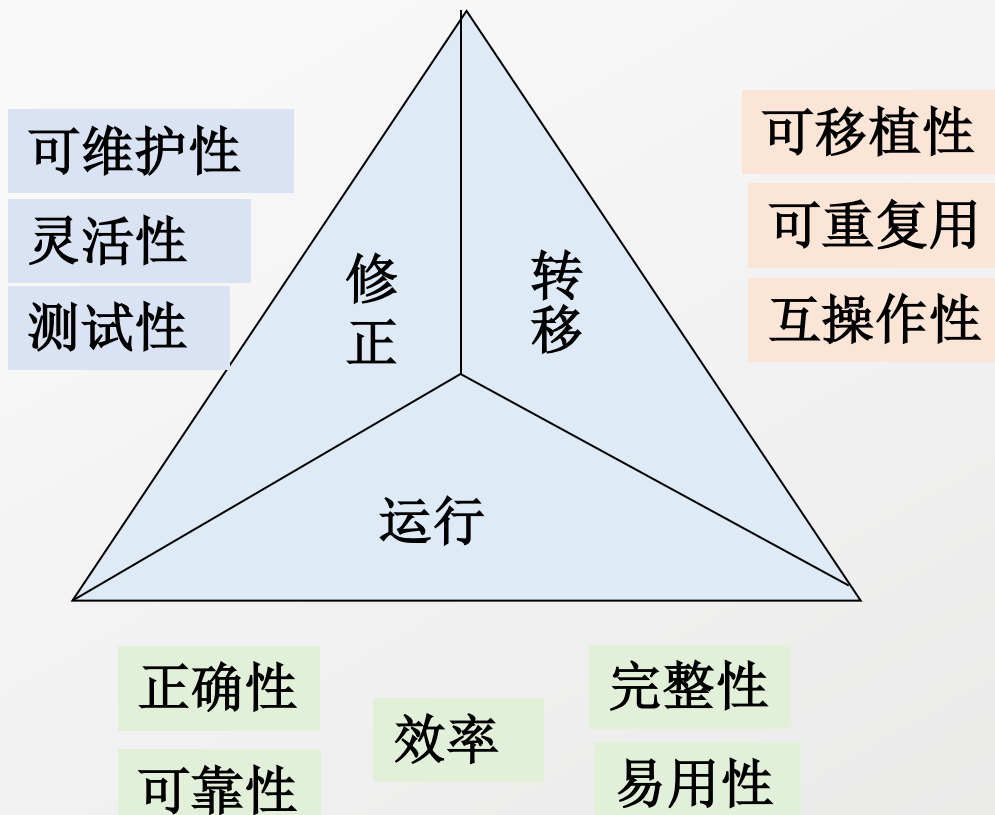
(1) 软件质量属性基本内容

用户对于已投入使用的软件，会关心软件运行以后表现出来的
运行特性、修正特性
和转移特性：

3个特性与11个
质量属性的关系



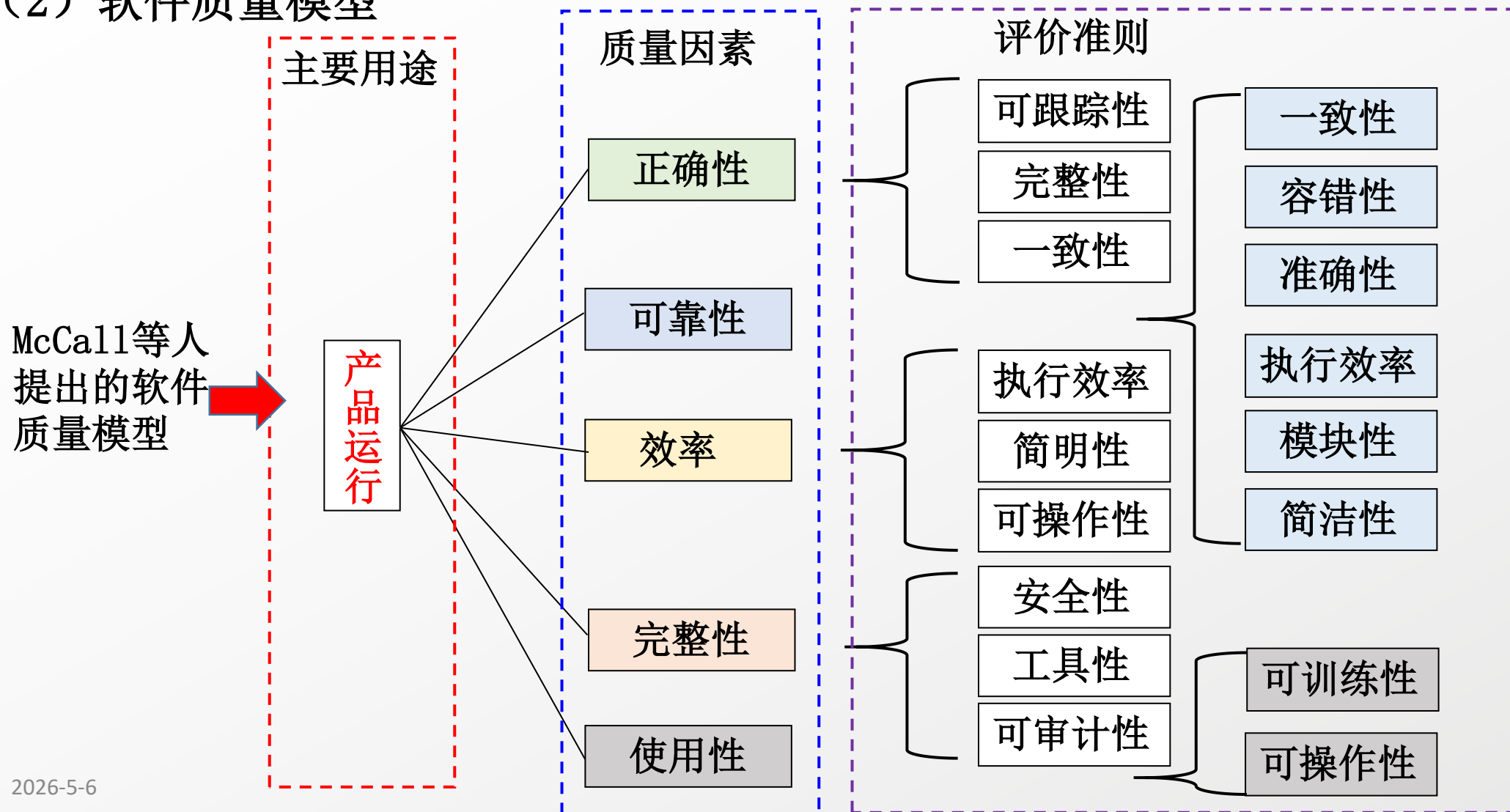
在运行阶段用户的质量特性观点



2.6 软件的质量属性

2.6.2 软件质量属性

(2) 软件质量模型

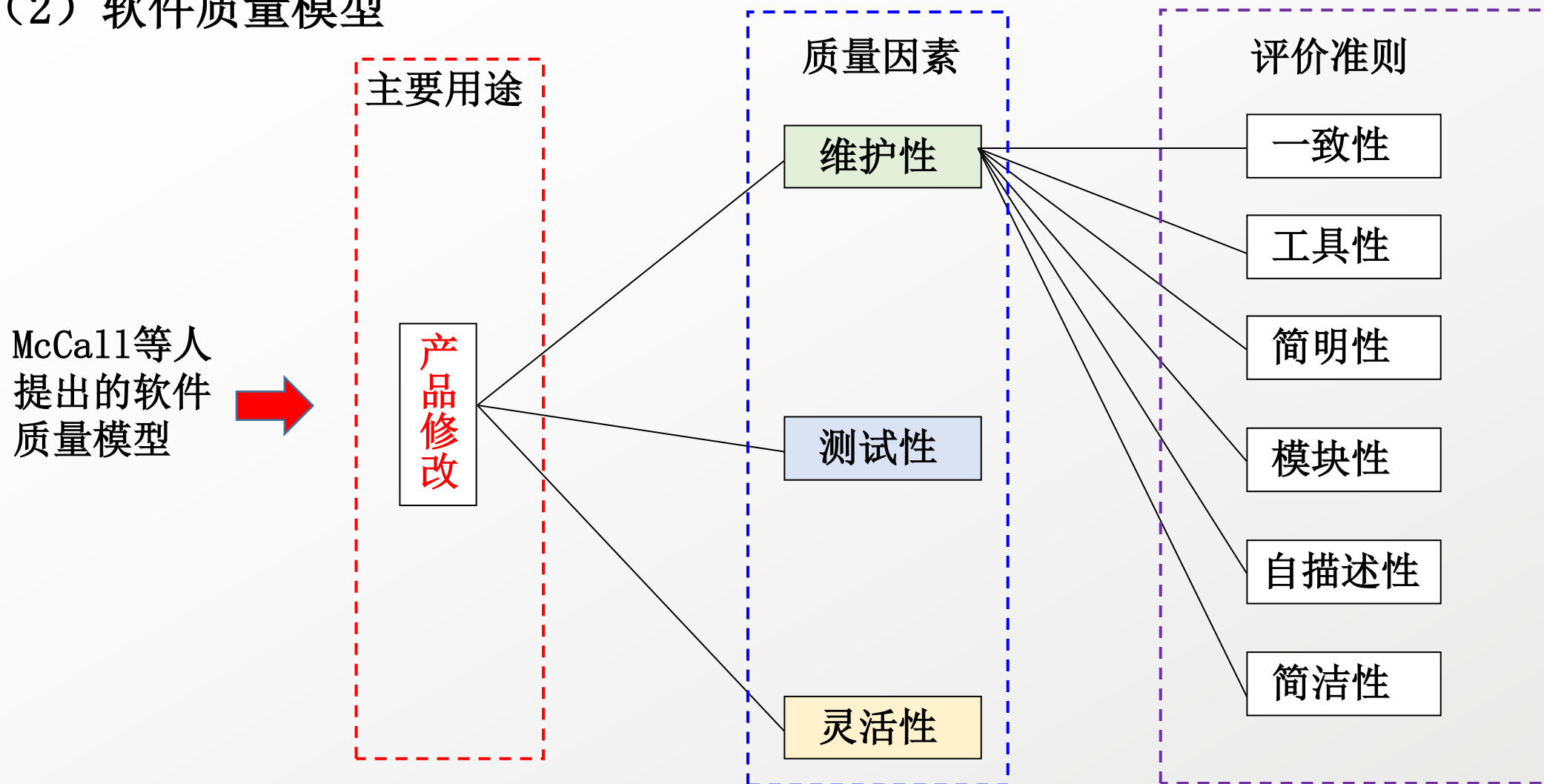




2.6 软件的质量属性

2.6.2 软件质量属性

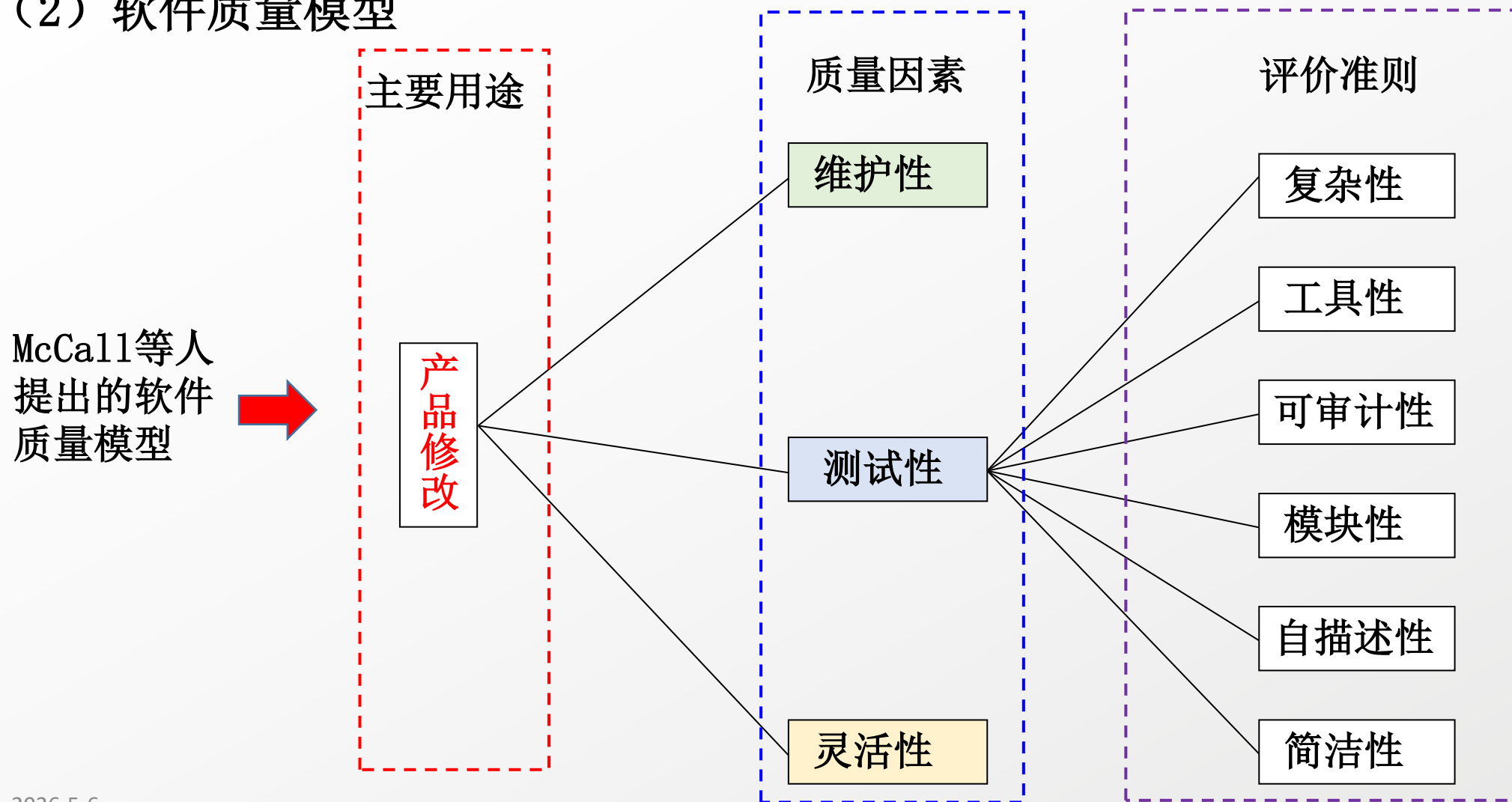
(2) 软件质量模型



2.6 软件的质量属性

2.6.2 软件质量属性

(2) 软件质量模型

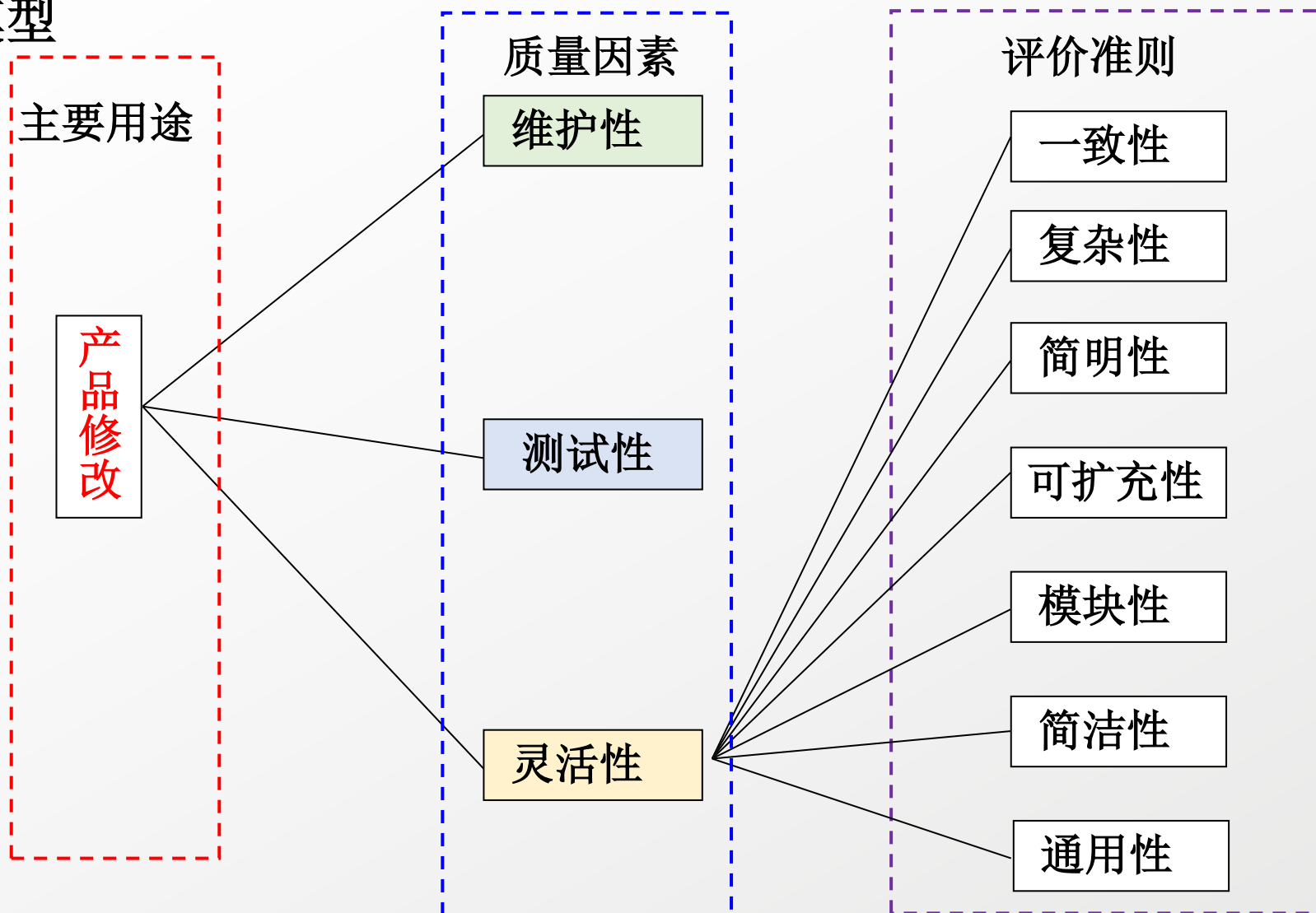
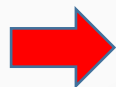


2.6 软件的质量属性

2.6.2 软件质量属性

(2) 软件质量模型

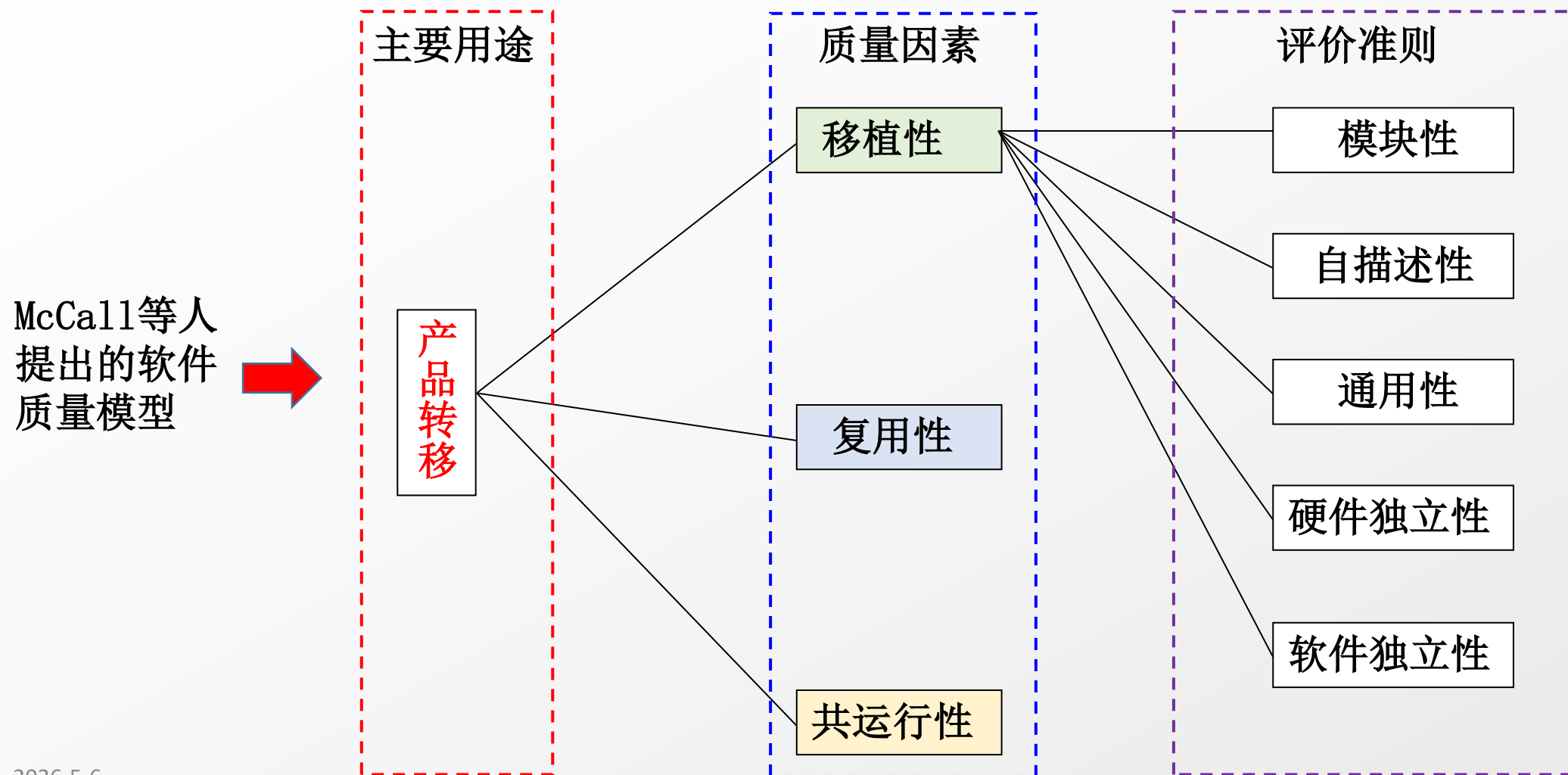
McCall等人
提出的软件
质量模型



2.6 软件的质量属性

2.6.2 软件质量属性

(2) 软件质量模型



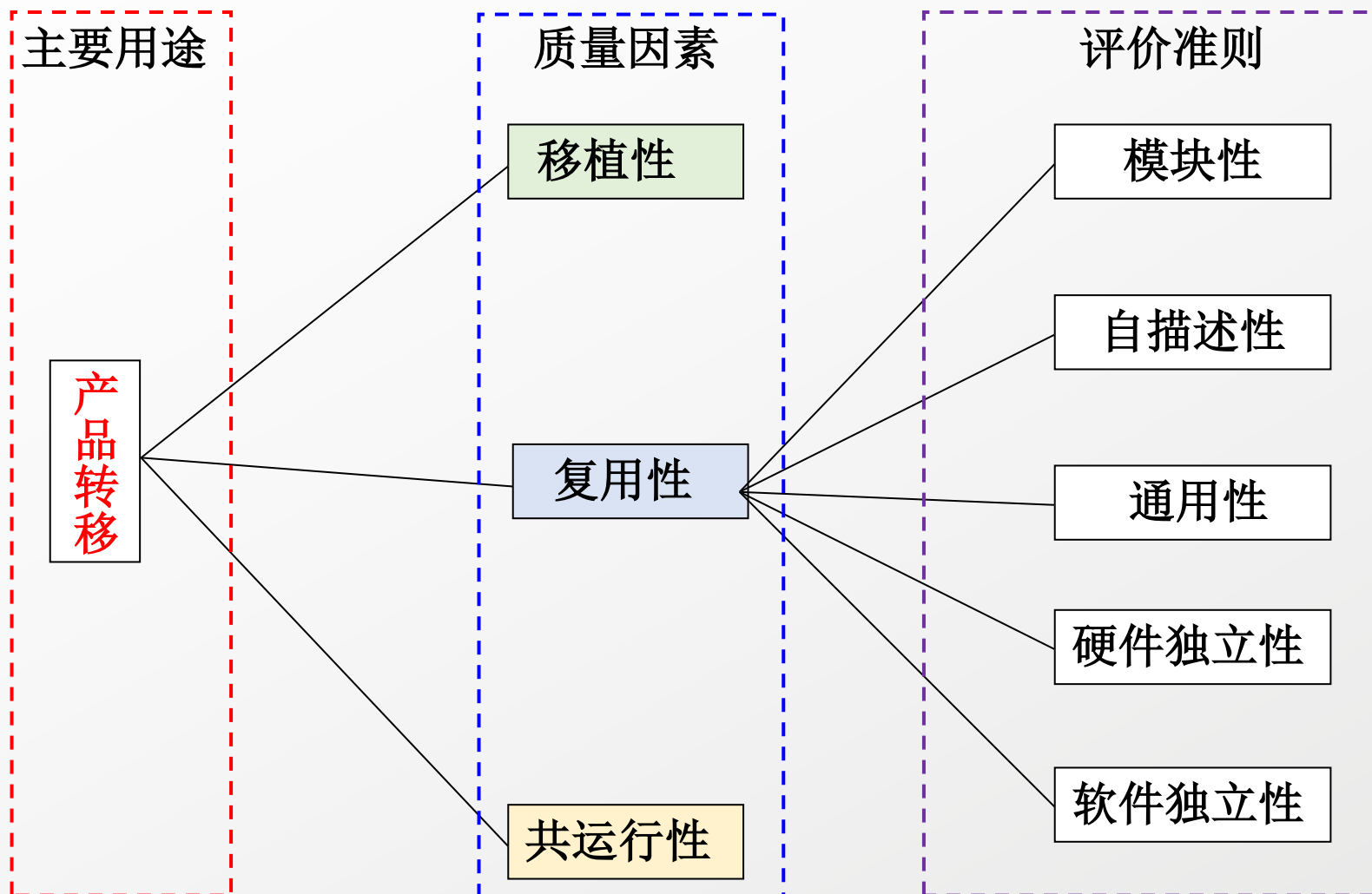
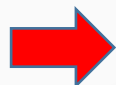


2.6 软件的质量属性

2.6.2 软件质量属性

(2) 软件质量模型

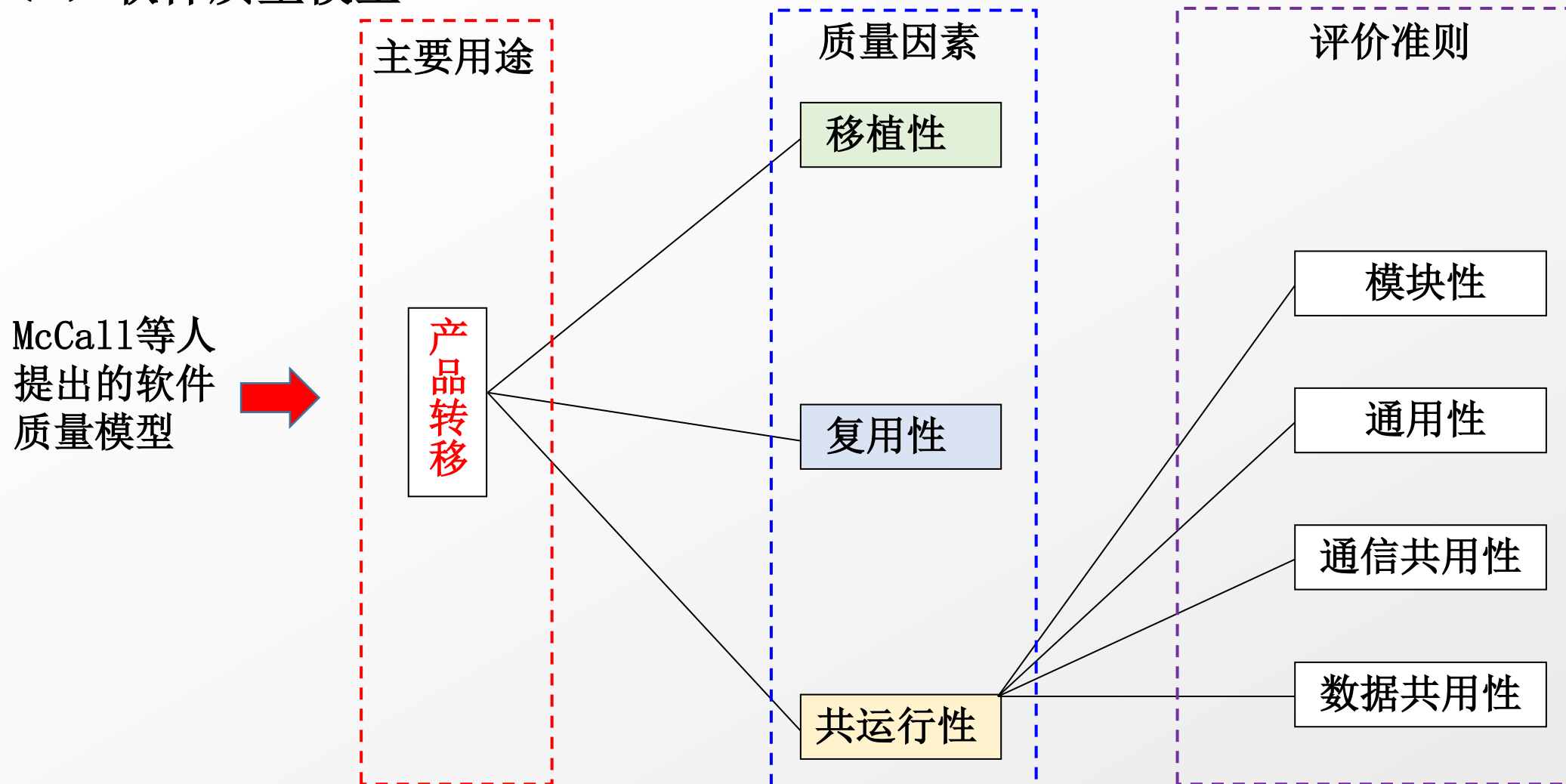
McCall等人
提出的软件
质量模型



2.6 软件的质量属性

2.6.2 软件质量属性

(2) 软件质量模型





2.6 软件的质量属性

2.6.2 软件质量属性

(3) 软件质量特性

序号	特性	含义
1	可跟踪性	沿一个设计说明或一个实际程序构件返回到需求的能力。
2	完整性	所需功能实现的程度。
3	一致性	在整个软件开发项目中使用统一的设计和文档编制技术的程度。
4	容错性	容错性
5	准确性	计算和控制的精确程度。
6	执行效率	程序的运行时间。
7	复杂性	程序结构化、模块化、简明、简洁、清晰和可理解的程度。



2.6 软件的质量属性

2.6.2 软件质量属性

(3) 软件质量特性

序号	特性	含义
8	安全性	控制或保护程序和数据机制的有效性。
9	工具性	程序监视自身运行和识别出现错误的程序。
10	简明性	程序代码的紧密程度。
11	可训练性	程序使新用户使用该系统的辅助程度。
12	可操作性	程序操作的难易程度。
13	可审计性	检查与标准是否符合的难易程度。
14	可扩充性	系结构、数据或过程设计可扩充的程度。
15	模块性	程序各构件的功能独立性。

2.6 软件的质量属性

2.6.2 软件质量属性

(3) 软件质量特性

序号	特性	含意
16	自描述性	源代码提供自身描述的程度。
17	简洁性	程序易于理解的程度。
18	通用性	程序各构件潜在的应用范围。
19	硬件独立性	软件与运行它的硬件之间的相关程度。
20	软件独立性	程序与非标准编程语言性质、操作系统特性、以及其他环境限制的无关程度
21	通信共用性	使用标准接口、协议和带宽的程度。
22	数据共用性	在整个程序中使用标准数据结构和类型的程度。

2.6 软件的质量属性

2.6.2 软件质量属性

(4) 国际标准

国际标准和国家标准规定的质量特性标准中规定了**6个质量特性**及其相关的**21个质量子特性**

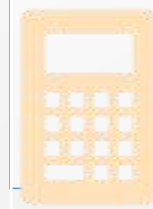
6个质量特性



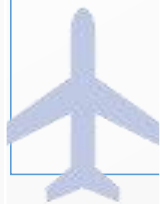
● 功能性



● 可靠性



● 易用性



● 效率



● 可维护性

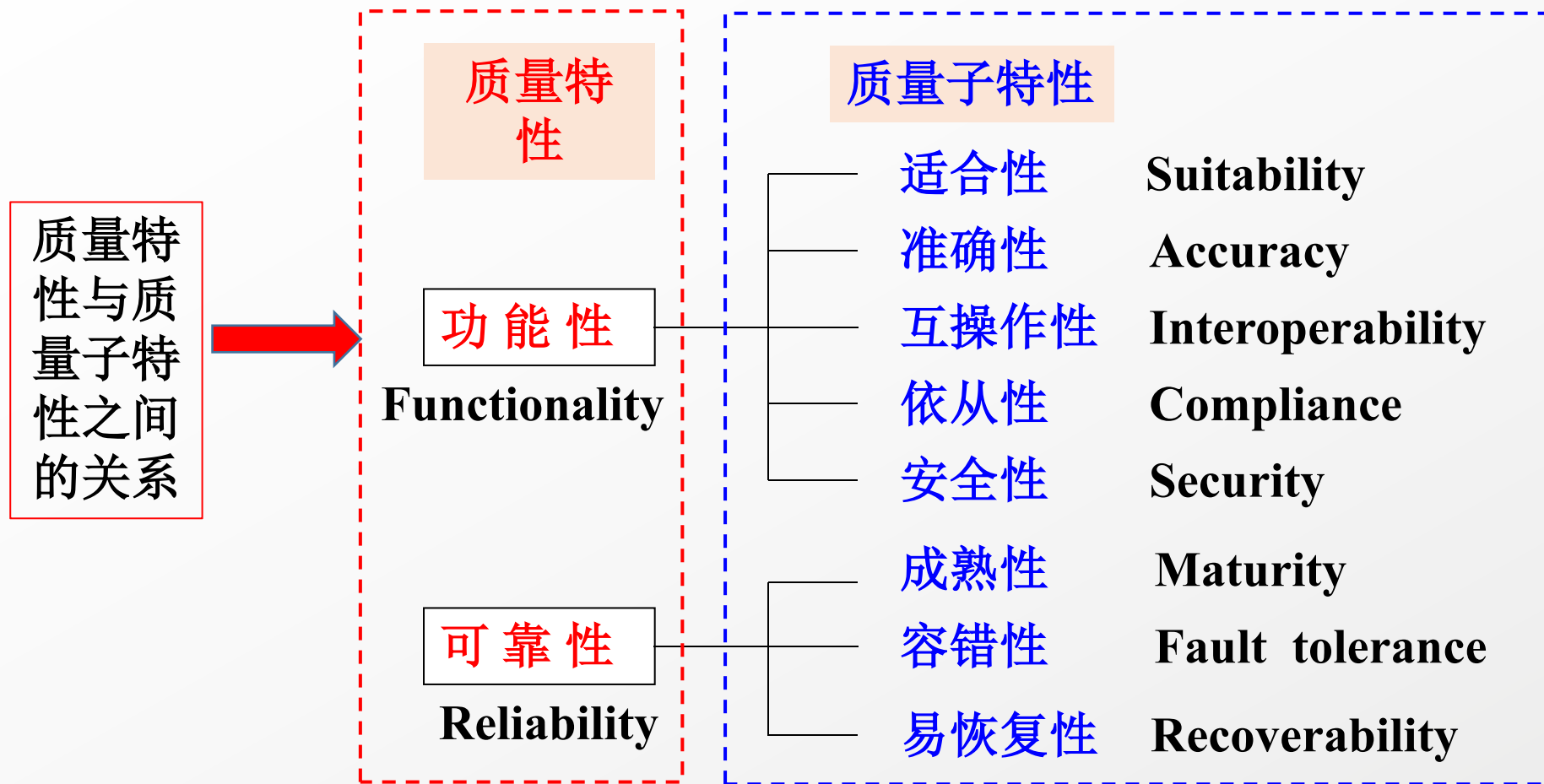


● 可移植性

2.6 软件的质量属性

2.6.2 软件质量属性

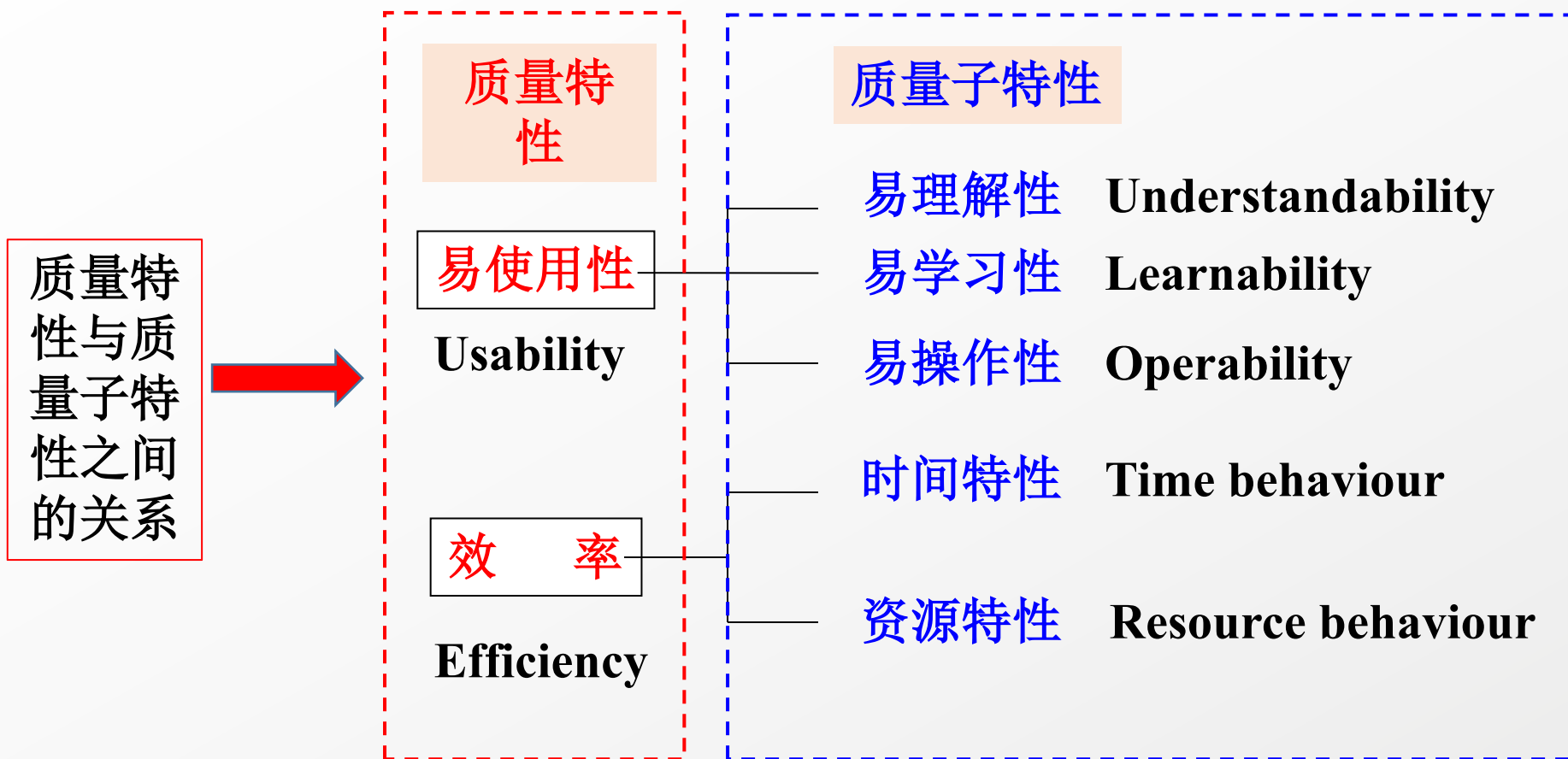
(4) 国际标准



2.6 软件的质量属性

2.6.2 软件质量属性

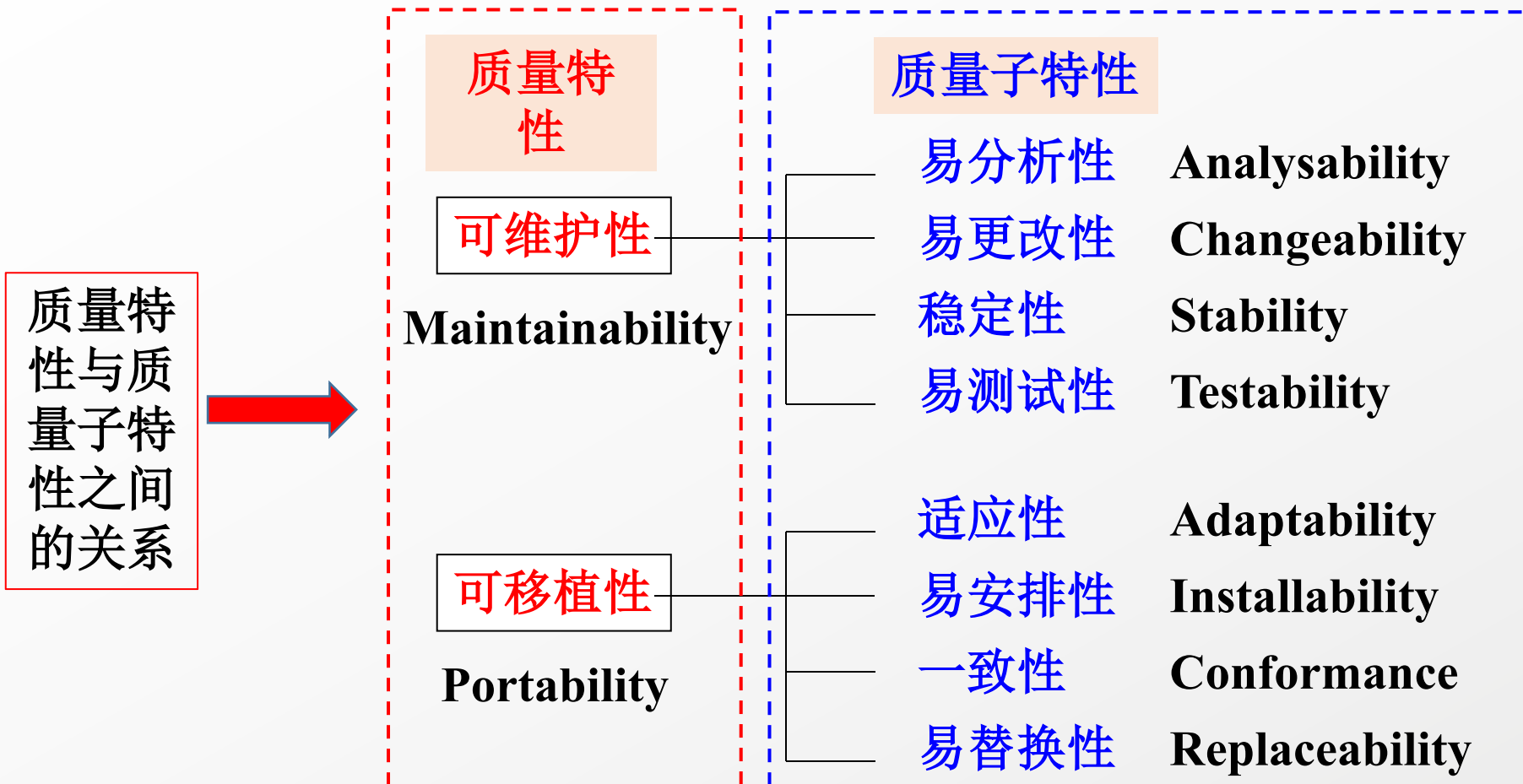
(4) 国际标准



2.6 软件的质量属性

2.6.2 软件质量属性

(4) 国际标准



2.6 软件的质量属性

2.6.2 软件质量属性

(4) 国际标准

质量特性	含意
功能性 Functionality	与一组功能及其制定的性质有关的一组属性。这里功能包括满足明确或隐含的要求。
可靠性 Reliability	与在规定的一段时间和条件下，软件能维持其性能水平的能力有关的一组属性。
易用性 Usability	与一组规定或潜在的用户为使用软件所需作的努力和对这样的使用所作的评价有关的一组属性。
效率 Efficiency	与在规定的条件下，软件的性能水平与所使用资源量之间关系有关的一组属性。
可维护性 Maintainability	与进行指定的修改所需的努力有关的一组属性。
可移植性 Portability	与软件可从某一环境移植到另一环境的能力有关的一组属性。

2.6 软件的质量属性

2.6.2 软件质量属性

(5) 不同视角的重要属性

对用户最重要的属性

可用性(availability)
有效性(efficiency)
灵活性(flexibility)
完整性(integrity)
互操作性(interoperability)
可靠(reliability)
健壮性(robustness)
易用性(usability)

对开发者最重要的属性

可维护性(maintainability)
可移植性(portability)
可重用性(reusability)
可测试性(testability)

2.6 软件的质量属性

2.6.2 软件质量属性

(5) 不同视角的重要属性

每个项目都要考虑的质量属性分为**2类**

对嵌入式系统很重要的属性(高效性和可靠性)

用于主机应用程序(有效性和可维护性)或桌面系统(互操作性和可用性)

理想环境下

每一个系统总是最大限度地展示所有这些属性的可能价值，系统将随时可用、决不会崩溃、可立即提供结果且易于使用

现实环境下

对用户最重要的属性

• 可用性、有效性、灵活性等

对开发者最重要的属性

• 可维护性、可移植性、可重用性等

其他属性

• 有效性、可维护性、互操作性等

定义用户和开发者的目标，使产品设计者做出合适选择

2.6 软件的质量属性

2.6.2 软件质量属性

(6) 定义方法1

- 1) 必须根据用户对系统的期望来确定质量属性
- 2) 定量地确定重要属性提供了对用户期望的清晰理解，有助于设计者提出最合理的解决方案

互操作性对你的重要性如何?
软件应该具有怎样的可靠性?



用户



分析员想出了对于不同的用户类可能很重要的属性，并根据每一个属性设计出许多问题



利用这些问题询问每一个用户类的代表，可以把每个属性分成一级(不必多加考虑的属性)到五级(极其重要的属性)

分析员



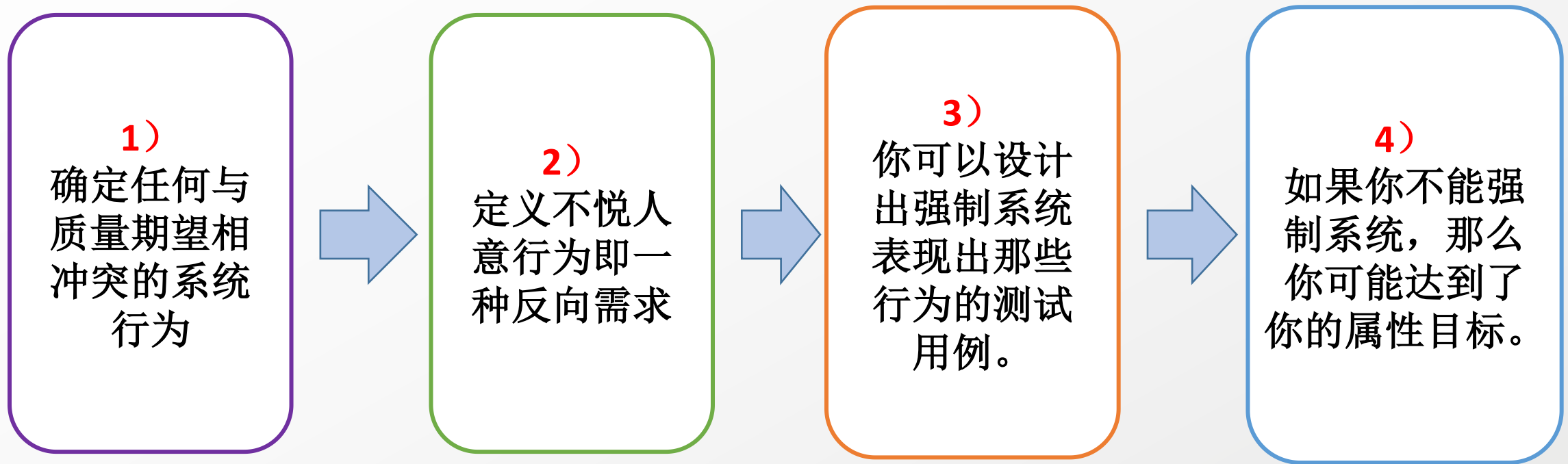
- 1) 为每一属性确定特定的、可测量的和可验证的需求
- 2) 如果质量目标不可验证，那么就说不清是否达到这些目标
- 3) 在合适的地方为每一个属性或目标指定级别或测量单位、最大和最小值
- 4) 如果不能定量地确定某些对项目很重要的属性，那么至少应该确定其优先级。

2.6 软件的质量属性

2.6.2 软件质量属性

(7) 定义方法2

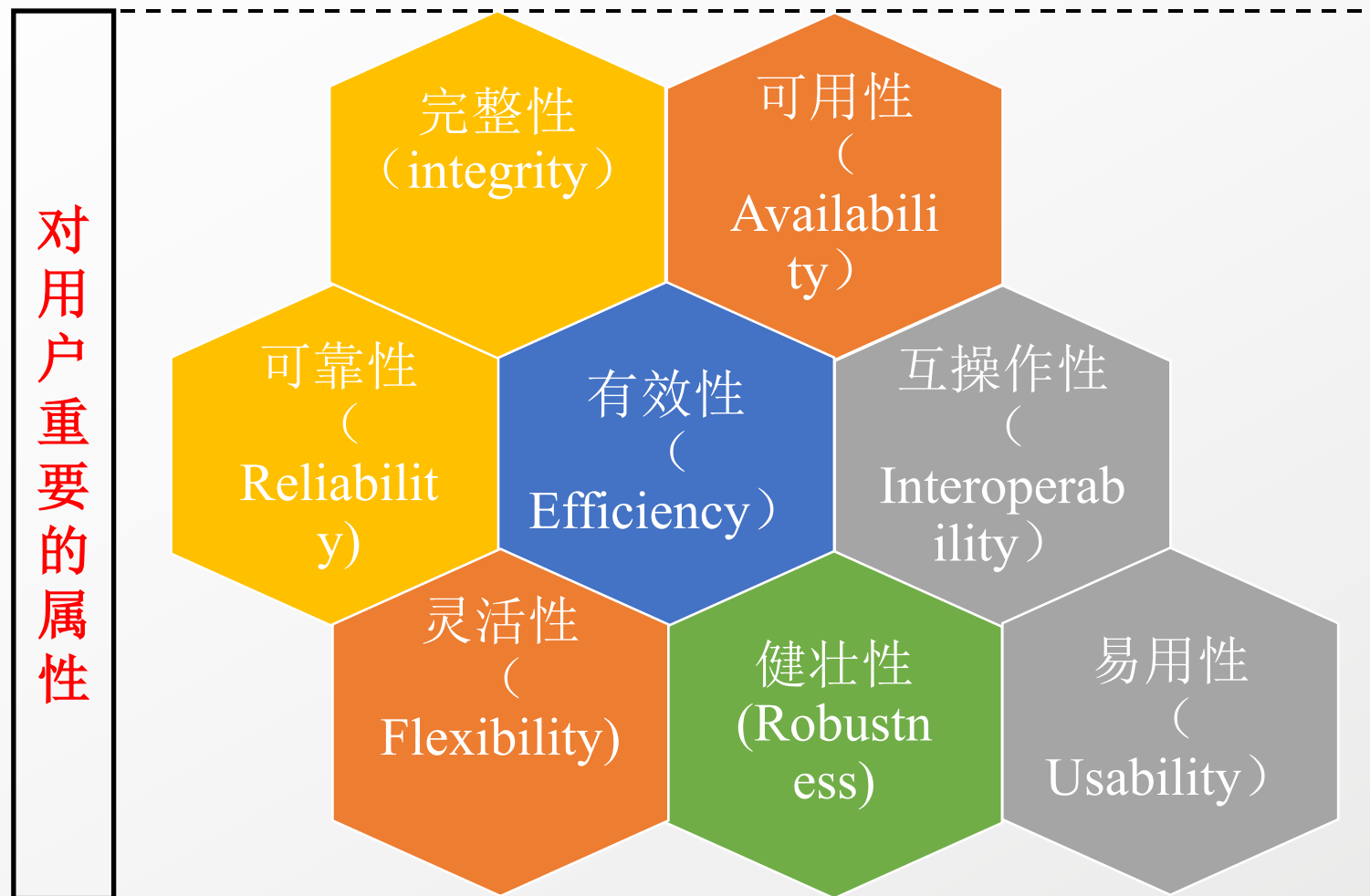
定义属性的方法2：确定任何与质量期望相冲突的系统行为



2.6 软件的质量属性

2.6.2 软件质量属性

(8) 属性的定义



2.6 软件的质量属性

2.6.2 软件质量属性

(8) 属性的定义—可用性

可用性：在预定的启动时间中，系统真正可用并且完全运行时间所占的百分比。

- 1) 有些任务比其它任务有更严格时间要求
- 2) 当要执行一个任务但系统在那一刻不可用时，用户会感到很沮丧

业务
安全目标
可用性



询问用户需要多高的可用性，并且是否在任何时间



一个可用性需求说明：

“工作日期间，在当地时间早上6点到午夜，系统的可用性至少达到99.5%，在下午4点到6点，系统的可用性至少可达到99.95%。”

2.6 软件的质量属性

2.6.2 软件质量属性

(8) 属性的定义—有效性

有效性：用来衡量系统在利用处理器的处理能力、磁盘空间或通信带宽等方面的能力。

1) 效率降低：如果系统用完了所有可用的资源，则用户将遇到性能的下降

2) 系统性能低：例如，一个实时处理系统超负荷，拙劣的系统性能可激怒等待数据库查询结果的用户，或者可能对系统安全性造成威胁

3) 实现在不可预料的情况下允许安全缓冲：“在预计的高峰负载条件下，10%处理器能力和15%系统可用内存必须留出备用。”



在定义性能、能力和效率目标时，考虑硬件的最小配置是很重要的

2.6 软件的质量属性

2.6.2 软件质量属性

(8) 属性的定义—灵活性

灵活性：也称可扩充性（extensibility）、可扩张性(augmentability)、可延伸性(extendability)和可扩展性(expandability)，用来测量向产品中增加新功能的难易程度、或所需工作量的大小。

1) 如果开发者预料到系统的扩展性，可选择合适的方法来最大限度地增大系统的灵活性

2) 灵活性对于通过一系列连续的发行版本，并采用渐增型和重复型方式开发的产品是很重要的

3) 例如：在一个图形工程中，灵活性目标是如下设定的：“一个至少具有6个月产品支持经验的软件维护程序员可以在一个小时之内为系统添加一个新的可支持硬拷贝的输出设备，包括代码修改和测试。”

2.6 软件的质量属性

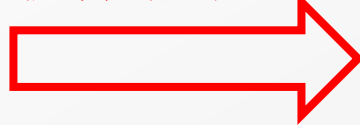
2.6.2 软件质量属性

(8) 属性的定义—完整性

完整性(或安全性): 主要包括防止非法访问系统功能、防止数据丢失、保护软件免受病毒入侵、保护输入到系统的数据的保密性和安全性，并防止私人数据进入系统。

- 1) 电子商务系统的用户关心的是保护信用卡信息
- 2) Web的浏览者不愿意那些私人信息或他们所访问过的站点记录被非法使用

完整性对通过WWW执行的软件非常重要



- 1) 完整性的需求不能犯任何错误，即数据和访问必须通过特定的方法完全保护起来
- 2) 用明确的术语陈述完整性的需求，如身份验证、用户特权级别、访问约束或需要保护的精确数据

例如：一个完整性的需求样本可以这样描述：“只有拥有查账员访问特权的用户才可以查看客户交易历史。”



2.6 软件的质量属性

2.6.2 软件质量属性

(8) 属性的定义—互操作性

互操作性：表明了产品与其它系统交换数据和服务的难易程度

1) 为了评估互操作性是否达到要求的程度，必须知道用户使用其它哪些应用程序与本产品协同工作，还要知道他们要交换什么数据。

2) 例如：

“化学制品跟踪系统”的用户习惯于使用一些商业工具绘制化学制品的结构图，所以他们提出如下的互操作性需求：**“化学制品跟踪系统应该能够从ChemiDraw和Chem-Struct工具中导入任何有效的化学制品结构图。”**

2.6 软件的质量属性

2.6.2 软件质量属性

(8) 属性的定义—可靠性

可靠性：是软件无故障执行一段时间的概率，**健壮性**和**有效性**有时可看成是可靠性的一部分。

衡量软件可靠性的方法

- 1) 正确执行操作所占的比例
- 2) 在发现新缺陷之前系统运行的时间长度和缺陷出现的密度
- 3) 若发生故障对系统的影响程度
- 4) 最大的可靠性的费用是否合理

满足高可靠性

- 1) 即使该软件还存在缺陷，也可认为达到其可靠
- 2) 高可靠性的系统也是为高可测试性系统设计的

例如：

一个用于控制实验室设备的软件，这些设备全天工作并且使用稀有的、昂贵的化学制品。用户要求真正与实验相关的那部分软件要高可靠性，而其它系统功能，例如周期性地记录温度数据，则对可靠性要求不高。

该系统的一个可靠性需求说明如下：“**由于软件失效引起实验失败的概率应不超过5%。**”

2.6 软件的质量属性

2.6.2 软件质量属性

(8) 属性的定义—健壮性

健壮性：是当系统或其组成部分遇到非法输入数据、相关软件或硬件组成部分的缺陷或异常的操作情况时，能继续正确运行功能的程度

- 1) 健壮的软件可以从发生问题的环境中完好地恢复并且可容忍用户的错误
- 2) 当从用户那里获取健壮性的目标时，询问系统可能遇到的错误条件并且要了解用户想让系统如何响应

例如：

一个健壮性需求是这样说明的：“所有的规划参数都要指定一个缺省值，当输入数据丢失或无效时，就使用缺省值数据。”这个例子反映了对一个“用户”是另一个软件应用程序的产品，其健壮性设计的方法。

2.6 软件的质量属性

2.6.2 软件质量属性

(8) 属性的定义—易用性 (Usability)

易用性：也称为“**ease of use**”和“人类工程”，它所描述的是许多组成“**用户友好**”的因素。

- 1) 易用性衡量了为准备输入、操作和理解产品输出所花费的努力
- 2) 必须权衡易用性和学习如何操纵产品的简易性

例子：

“化学制品跟踪系统”的分析员询问用户：“**快速、简单地请求化学制品并浏览其它信息，这一需求对你有多重要？**”、“**你请求一种化学制品大概需花多少时间？**”

3) 对于可用性的讨论可以得出可测量的目标，例如：

“一个培训过的用户应该可以在平均**3分钟**或最多**5分钟**时间以内，完成从供应商目录表中请求一种化学制品的操作。”

以前从没有使用过“化学制品跟踪系统”的化学家，在经过最多不超过**30分钟**的适应之后，应该能正确地对一个化学品提出请求。



2.6 软件的质量属性

2.6.2 软件质量属性

(8) 属性的定义—易用性 (Usability) (续)

4) 易用性还包括对于新用户或不常使用产品的用户在学习使用产品时的简易程度

易学程度的目标可以经常定量地测量，例如：

“一个新用户用不到30分钟时间适应环境后，就应该可以对一个化学制品提出请求”，或者“新的操作员在一天的培训学习之后，就应该可以正确执行他们所要求的任务的95%”

5) 当定义易用性或易学性的需求时，应考虑在判断产品是否达到需求而对产品进行测试的费用

2.6 软件的质量属性

2.6.2 软件质量属性

(8) 属性的定义--可维护性 (Maintainability)

可维护性：表明了软件中纠正一个缺陷或做一次更改的简易程度

可维护性**取决于理解软件、更改软件和测试软件的简易程度**，可维护性与灵活性密切相关

- 1) 高可维护性对于那些经历周期性更改的产品或快速开发的产品很重要
- 2) 你可以根据修复(fix)一个问题所花的平均时间和修复正确的百分比来衡量可维护性

例子1:

“化学制品跟踪系统”包括如下的可维护性需求：“在接到来自联邦政府修订的化学制品报告的规定后，对于现有报表的更改操作必须在一周内完成。”

例子2:

在图形引擎工程中，须不断更新软件以满足用户需要，可维护性描述如下：“**函数调用不能超过两层深度**”，并且“**每一个软件模块中，注释与源代码语句的比例至少为1：2**”

2.6 软件的质量属性

2.6.2 软件质量属性

(8) 属性的定义—可移植性 (Portability)

可移植性：是度量把一个软件从一种运行环境转移到另一种运行环境中所花费的工作量

- 1) 软件可移植的设计方法与软件可重用的设计方法相似
- 2) 可移植性对于工程的成功是不重要的，对工程的结果也无关紧要
- 3) 可以移植的目标必须陈述产品中可以移植到其它环境的那一部分，并确定相应的目标环境



依据

开发者选择设计和编码方法以适当提高产品可移植性

2.6 软件的质量属性

2.6.2 软件质量属性

(8) 属性的定义—可重用性(Reusability)

可重用性：是软件开发的长远目标，表明把一个软件组件用于其他应用程序所涉及的相关工作量。

1) 比起创建一个打算只在一个应用程序中使用的组件，**开发可重用软件的费用会更大些。**

2) 可重用软件必须**标准化、模块化、文档资料齐全、不依赖于特定的应用程序和运行环境、并具有通用性。**

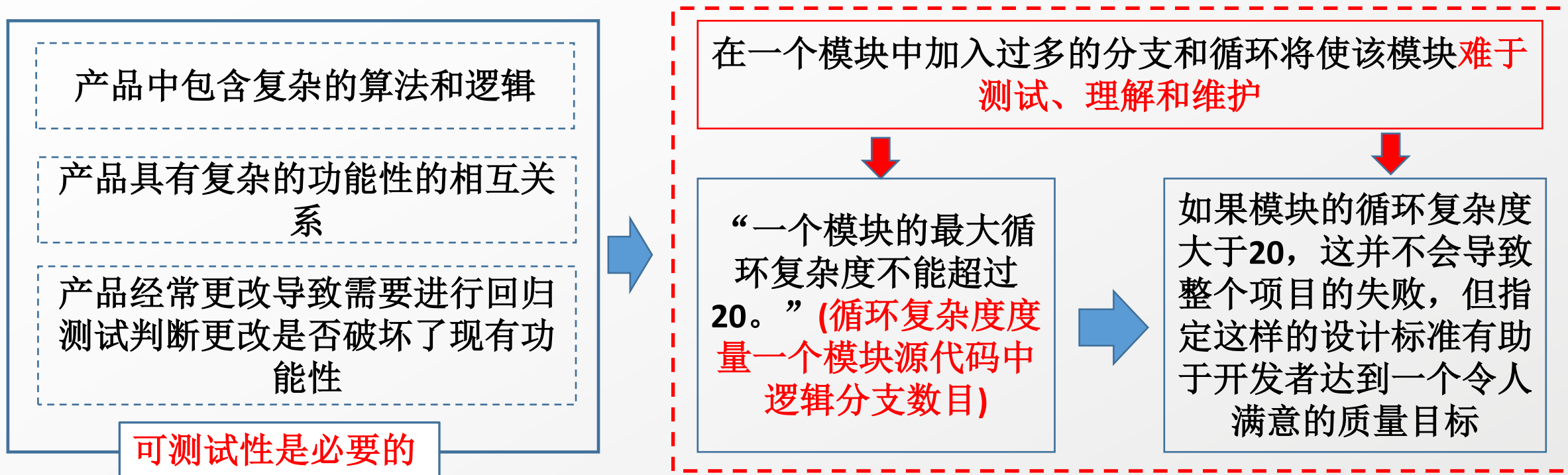
3) 可重用性目标难以进行定量描述：确定新系统中哪些元素需要用方便于代码重用的方法设计，或者规定作为项目副产品的可重用性组件库。

2.6 软件的质量属性

2.6.2 软件质量属性

(8) 属性的定义—可测试性 (Testability)

可测试性：也称为**可验证性 (verifiability)**，指的是测试软件组件或集成产品时查找缺陷的简易程度。



2.6 软件的质量属性

2.6.2 软件质量属性

(9) 性能需求

性能需求：定义了系统必须多好地完成专门的功能，包括**速度**（如数据库响应时间）、**吞吐量**（每秒钟处理的事务）、**处理能力**（并发使用负载）和**定时**（严格的实时要求）。

注意：苛刻的性能需求会对设计软件策略和选择硬件造成严重的影响。因此，定义的性能需求目标要适合于运行环境。

性能需求的例子：

- 1) 温度控制循环必须在**80毫秒**内完成执行；
- 2) 解释器每分钟应该至少解析**5000**条没有错误的语句；
- 3) 在通过**100KBps**的调制解调器与Internet相连的情况下，下载一个页面需要**10秒**或更短；
- 4) **ATM**自动柜员机系统对提款请求的身份认证不能超过**10秒**。

2.6 软件的质量属性

2.6.2 软件质量属性

(10) 非功能需求

- 1) 设计人员和编程人员必须确定最好的方法来满足每一个质量属性和性能需求。
- 2) 尽管质量属性是非功能需求，但能导致衍生的功能需求、设计原则或其他类型的技术信息，这些信息将产生期望的质量特性。

 完整性、互操作性、健壮性、易用性和安全防护性

可能转化为

功能性需求类技术信息

 可用性、有效性、灵活性、操作性和可靠性

可能转化为

系统架构类技术信息

 互操作性、易用性

可能转化为

设计限制类技术信息

 灵活性、可维护性、可靠性、可重用性、可测试性和易用性

可能转化为

设计原则类技术信息

 可移植性

可能转化为

实现限制类技术信息

2.6 软件的质量属性

2.6.2 软件质量属性

(11) 属性的折中方案

- 1) 有时，不可避免地要对一些特定的属性对进行取舍。用户和开发者必须确定哪些属性比其它属性更为重要，并定出优先级。在他们作决策时，要始终遵照这些优先级。
- 2) **某质量属性的增强会对其它的属性产生积极或不利影响。**例如，增强软件可重用性的设计方法也可以使软件变得灵活、更易于与其它软件组件相连接、更易于维护、更易于移植并且更易于测试。
- 3) **高效性对其它许多属性具有消极影响。**如果你编写最紧凑，最快的这将不易移植到其它环境，而且还难于维护和改进软件代码，并使用一种特殊的预编译器和操作系统，软件就难以移植。
- 4) 一些优化操作者易用性的系统或企图具有灵活性、可用性并且可以与其它软硬件相互操作的系统将**付出性能方面的代价**。例如，比起使用具有完整的制定图形代码的旧应用系统，使用外部的通用图形引擎工具生成图形规划将大大降低性能。你必须在性能代价和你所提出的解决方案的预期利益之间作出权衡，以确保作出合理的取舍。

2.6 软件的质量属性

2.6.2 软件质量属性

(11) 属性的折中方案

- 5) 如果软件要在多平台下运行(可移植性), 那么易用性一般不好。
- 6) 可重用软件能普遍适用于多种环境中, 因此, 不能达到特定的容错(可靠性)或完整性目标。
- 7) 对于高安全的系统, 很难完全测试其完整性需求;
- 8) 可重用的类组件或与其它应用程序的互操作可能会破坏其安全机制。

在需求获取的过程中, 加入对质量属性期望的讨论, 并把所了解的写入软件需求规格说明中, 才有可能实现质量特性的合理平衡

当你为项目定义重要的质量属性时, 要防止发生与目标冲突的行为。例如: 如果软件要在多平台下运行(可移植性), 那么易用性一般不好。

需求获取阶段识别

确定优先级

确定相关的质量属性

产品特性达到最佳平衡

2.6 软件的质量属性

2.6.2 软件质量属性

(12) 质量属性举例—性能需求

性能
需求

●R1: 在最大负荷时, 产品应能处理100项支付交易/每秒。

●R2: 产品应能在0.02秒内处理一项警告。

R3: 在标准工作负荷时, 产品的CPU使用率应少于50%, 其余50%应留给后台作业。

R4: 200页文档的前后翻页不应超过1秒, 搜索一关键字不应超过5秒。

R5: 切换至下一字段时, 应能在0.2秒内做输入; 切换至下一屏幕时, 应能在1.2秒内做输入。

R6: 显示报表时, 95%的简单报表应少于20秒, 任何报表都不应该超过80秒。

2.6 软件的质量属性

2.6.2 软件质量属性

(12) 质量属性举例—安全性需求

R1

该产品应预防数据库的丢失。预计的损失应每5年小于1。

R2

该产品应预防磁盘发生故障。预计的损失应每10年小于1。

R3

该产品应采用磁盘阵列(RAID)。

R4

该产品应防止病毒删除文件, 这类病毒攻击的其他风险应小于xx。

R5

该产品应包含能够防止病毒的防火墙。

R6

该产品应防止数据传输至会计系统之前删除发票数据。

2.7 需求获取小结

(1) 需求获取的3个层次

需求获取(requirement elicitation)是需求工程的主体

第一层：业务需求

来自项目视图和范围文档的业务需求决定用户需求，它描述了用户利用系统需要完成的任务。

第二层：用户需求

获取用户需求位于软件需求三层结构的中间层

第三层：功能需求

分析者能获得用于描述系统活动的特定的软件功能需求，以便用户执行任务

2.7 需求获取小结

(2) 需求获取的要点

需求获取的重点是项目视图和范围确定、选择用户和产品代表、使用实例方法的应用



1) 需求获取是在问题及其最终解决方案之间架设桥梁的第一步



2) 获取需求的一个必不可少的结果是对项目中描述的客户需求的普遍理解



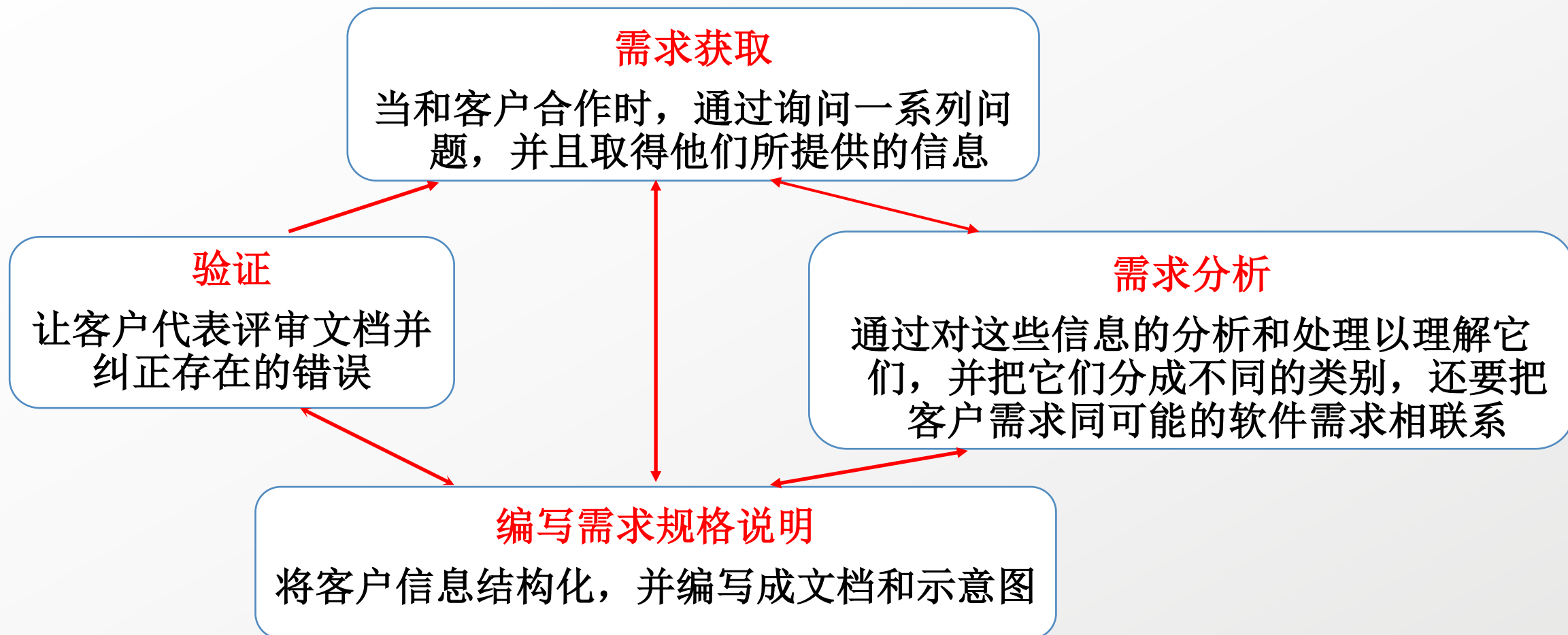
3) 一旦理解了需求，分析者、开发者和客户就能探索出描述这些需求的多种解决方案



4) 参与需求获取者只有在他们理解了问题之后才能开始设计系统，否则，对需求定义的任何改进，设计上都必须大量的返工

2.7 需求获取小结

(3) 需求开发中4个过程及其相互关系



2.7 需求获取小结

(4) 需求获取的指导方针

需求获取可能是软件开发中最困难、最关键、最易出错及最需要交流的方面：

- 1) 需求获取只有通过有效的客户——开发者的合作才能成功
- 2) 分析者必须建立一个对问题进行彻底探讨的环境，而这些问题与产品有关
- 3) 为方便清晰地进行交流，就要列出重要的小组，而不是假想所有的参与者都持有相同的看法
- 4) 对需求问题的全面考察需要一种技术，利用这种技术不但考虑了问题的功能需求方面，还可讨论项目的非功能需求。
- 5) 确定用户已经理解：对于某些功能的讨论并不意味着即将在产品中实现它。对于想到的需求必须集中处理并设定优先级以避免一个不能带来任何益处的无限扩大范围的项目。

2.7 需求获取小结

(4) 需求获取的指导方针

6) 需求获取是一个需要高度合作的活动，而并不是客户所说的需求的简单拷贝

7) 一个分析者必须透过客户所提出的表面需求理解他们的真正需求。询问一个可扩充 (open-ended) 的问题有助于你更好地理解用户目前的业务过程并且知道新系统如何帮助或改进他们的工作

8) 调查用户任务可能遇到的变更，或者用户需要使用系统其它可能的方式

9) 想像你自己在学习用户的工作，你需要完成什么任务？你有什么问题？从这一角度来指导需求的开发和利用

10) 探讨例外的情况：

a. 什么会妨碍用户顺利完成任务？

b. 对系统错误情况的反映，用户是如何想的？

c. 询问问题时，以“还有什么能.....”，“当.....时，将会发生什么”，“你有没有曾经想过.....”，“有没有人曾经.....”为开头。记下每一个需求的来源，这样向下跟踪直到发现特定的客户。

2.7 需求获取小结

(4) 需求获取的指导方针

11) 尽量把客户所持的假设解释清楚，特别是那些发生冲突的部分

12) 从字里行间去理解以明确客户没有表达清楚但又想加入的特性或特征

13) 使用“上下文无关问题”：这是一个高层次的问题，它可以获取业务问题和可能的解决方案的全部信息。

例如：

客户对这些问题的回答“产品要求怎样的精确”或“你能帮我解释一下你为什么不同意某人的回答吗？”可以更直接地认识问题，而这是封闭(close—end)问题所不能做到的



2.7 需求获取小结

(5) 需求获取的典型来源

软件需求的7种典型来源：

- 1) 访问并与有潜力的用户探讨
- 2) 把对目前的或竞争产品的描述写成文档
- 3) 系统需求规格说明
- 4) 对当前系统的问题报告和增强要求
- 4) 市场调查和用户问卷调查
- 6) 观察正在工作的用户
- 7) 用户任务的内容分析



2.7 需求获取小结

(5) 影响项目成功最主要因素中的需求获取方面

研究表明：比起不成功的项目，一个成功的项目在开发者和客户之间采用了更多的交流方式：

1) 与单个客户或潜在的用户组一起座谈，对于业务软件包或信息管理系统(MIS)的应用来说是一种传统的需求来源。

2) 直接聘请用户进行获取需求的过程是为项目获得支持和买入 (buy-in) 的一种方式

3) 在每一次座谈讨论之后，记下所讨论的条目(item)，并请参与讨论的用户评论并更正

4) 及早并经常进行座谈讨论是需求获取成功的一个关键途径，因为只有提供需求的人才能确定是否真正获取需求

5) 进行深入收集和分析以消除任何冲突或不一致性

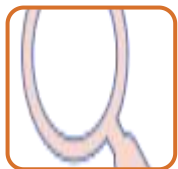


2.7 需求获取小结

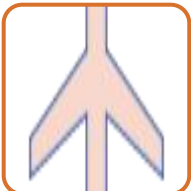
(5) 影响项目成功最主要因素中的需求获取方面



- 1) 尽量理解用户用于表述他们需求的思维过程
- 2) 充分研究用户执行任务时作出决策的过程，并提取出潜在的逻辑关系
- 3) **流程图**和**决策树**是描述这些**逻辑决策途径**的好方法



- 1) 当进行需求获取时，应避免受不成熟的细节的影响
- 2) 在对切合的客户任务取得共识之前，用户能很容易地在一个报表或对话框中列出每一项的精确设计
- 3) 若将细节都作为需求记录下来，会给随后的设计过程带来不必要的限制

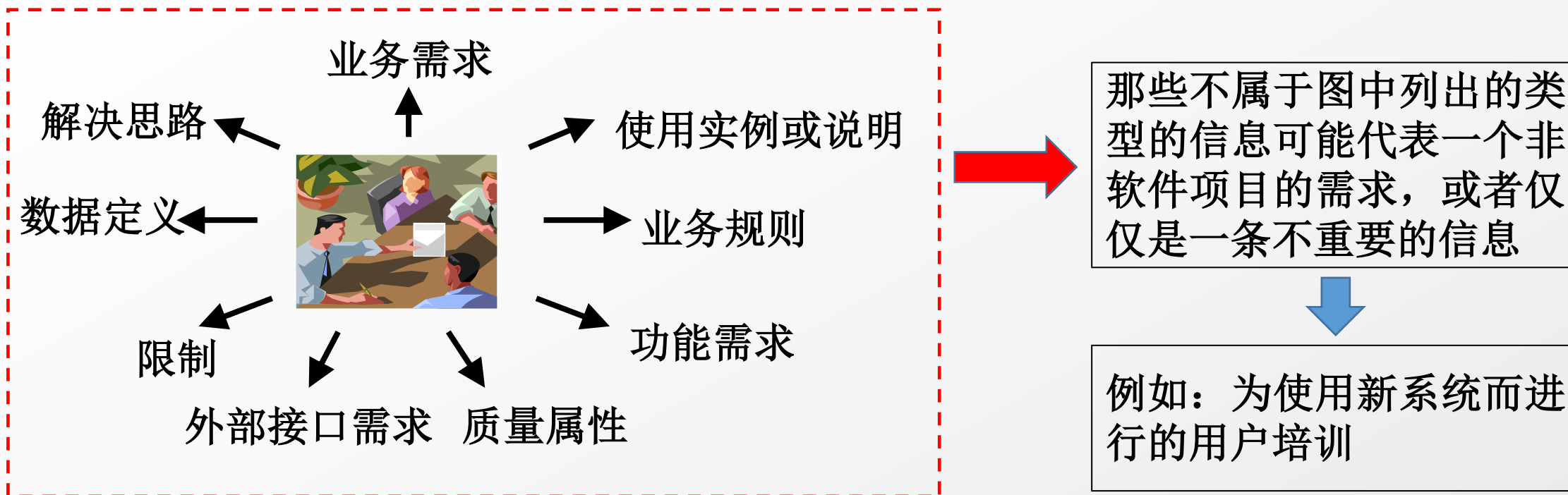


- 1) 在一个逐次详细描述过程中，重复地详述需求，以确定用户目标和任务，并作为使用实例。
- 2) 把任务描述为功能需求，这些功能需求可以使用户完成其任务；也可以把它们描述成非功能需求，这些非功能需求描述了系统的限制和用户质量的期望

2.7 需求获取小结

(6) 用户需求分类

- 1) 不要指望你的客户会给需求分析者提供一个简洁、完整、组织良好的需求清单
- 2) 分析者必须把代表客户需求的许多信息分成不同的类型，这样才能合理地编写文档



2.7 需求获取小结

(6) 用户需求分类--业务需求、用例或说明、业务规则

听取客户需求过程中的一些**建议**以帮助对信息进行分类处理：

1) 业务需求

- 描述客户或公司可从产品中得到的资金、市场或其它业务利润的需求是**最可能的业务需求**
- **倾听直接或间接的软件用户得到的好处**：“市场股票价格上升x%”、“每年节约\$Y”、“替代了维护费用高的老一代系统Z”

2) 用例或说明

- **用例**：有关利用系统执行的业务任务或达到用户目标的总的陈述，**特定的任务描述表示用法说明**
- **可通过让客户描述其业务 workflow 活动来获取用例，可与客户商讨把特定任务概括成更广泛的用例**

3) 业务规则

- 当一个客户说，一些活动只能在特定的条件下，由一些特定的人来完成时，该用户可能在描述一个**业务规则**，例如：“如果一个药剂师在危险化学制品培训方面是可靠的，那么他就可以在一级危险药品清单上订购化学制品”
- **业务规则是有关业务过程的操作原则**，可以用一些软件功能需求来加强规则，例如：让“化学制品跟踪系统”可以访问培训记录数据库

2.7 需求获取小结

(6) 用户需求分类—功能需求、质量属性

4) 功能需求

- **最可能的功能需求**: 客户所说的“用户应该能<执行某些功能>”、“系统应该<具备某些行为>”
- **功能需求**描述了系统所展示的可观察的行为, 并且大多数是处于执行者——系统响应顺序的环境中
- **功能需求**定义了系统应该做什么, 它们组成了软件需求规格说明的一部分
- **分析者**应该明确, 每个人应理解系统为什么“必须”执行某一功能, 所提出的功能需求有时反映了过时的或无效的业务过程, 而这些过程不能加入到新系统中

5) 质量属性

- **质量属性**: 对系统如何能很好地执行某些行为或让用户采取某一措施的陈述, 是一种非功能需求
- **听取那些描述合理特性的意见**: 快捷、简易、直觉性、用户友好、健壮性、可靠性、安全性和高效性
- 需要和用户一起商讨精确定义他们**模糊的和主观言辞的真正含义**

2.7 需求获取小结

(6) 用户需求分类—外部接口需求、限制

6) 外部接口需求

- 外部接口需求描述了系统与外部的联系
- 软件需求规格说明必须包括用户接口和通信机制、硬件和其它软件系统需求部分
- 客户描述外部接口需求包括如下习惯用语：“从<某些设备>读取信号”、“给<一些其它系统>发送消息”、“以<某种格式>读取文件”、“能控制<一些硬件>”

7) 限制

- 限制：指一些合理限制设计者和程序员选择的条件，代表了另一种类型的非功能需求，必须把这些需求写入软件需求规格说明
- 一定的限制有助于提高产品质量属性，但应尽量防止客户施加不必要的限制，因为这将妨碍提出一个好的解决方案、会降低利用现有商业化软件集成解决方案的能力
- 只利用程序语言的标准命令而不允许使用供应商的扩展，可以提高可移植性
- 下面是客户描述限制的一些习惯用语：“必须使用<一个特定的数据库产品或语言>”、“不能申请多于<一定数量的内存>”、“操作必须与<其它系统>相同”、“必须与<其它应用程序>一致”

2.7 需求获取小结

(6) 用户需求分类—数据定义、解决思想

8) 数据定义

- 当客户描述一个数据项或一个复杂的业务数据结构的格式、允许值或缺省值时，他们正在进行**数据定义**，例如：“邮政编码由5个数字组成，后跟一个可选的短划线或一个可选的四位数字，缺省为0000”就是一个数据定义
- 需要把**数据定义集中在一个数据字典中**，作为项目参与者在整个项目开发和维护中的主要参考文档

9) 解决思想

- 若客户描述了用户与系统交互的特定方法，以使系统产生一系列活动(如：用户从下载清单中选择一个所需要的项)，则这是一个**建议性的解决方案**，而非需求，会导致获取需求小组成员在潜在的真正需求上分散精力
- 在获取需求时，应该把**重点放在需要作什么**而不是新系统应该如何设计和构造。
- 探讨客户为什么提出一个特定的实现方法，这可以帮助你理解真正的需求和用户如何构造系统的隐含的期望

2.7 需求获取小结

(7) 需求获取中注意问题--产品范围的定义 (1/3)

1) 产品范围的定义

在需求获取的过程中，你可能会发现对存在误差，不是太大就是太小

如果范围太大，你将要收集比真正需要更多的需求，以传递足够的业务和客户的值，此时获取过程将会拖延

如果项目范围太小，那么客户将会提出很重要的但又在当前产品范围之外的需求。当前的范围太小，以致不能提供一个令人满意的产品

需求的获取将导致修改项目的范围和任务，但作出这样具有深远影响的改变，一定要小心谨慎

2.7 需求获取小结

(7) 需求获取中注意问题——需求的任务 (2/3)

2) 需求主要是确定系统做什么的问题

“需求主要是关于系统做什么，而解决方案如何实现是属于设计的范围”的说法虽然简洁，但似乎过于简单化

需求的获取应该把重点放在“做什么”上，但在分析和设计之间还是存在一定的距离

可以使用假设“怎么做”来分类并改善对用户需求的理解。

在需求的获取过程中，分析模型、屏幕图形和原型可以使概念表达得更加清楚，然后提供一个寻找错误和遗漏的办法

把在需求开发阶段所形成的模型和屏幕效果看成是方便高效交流的概念性建议，而不应该看成是对设计者选择的一种限制

2.7 需求获取小结

(7) 需求获取中注意问题——选择适当数量的需求分析者和用户代表 (3/3)

3) 需求分析者和用户代表的数量

需求获取中如果参与者过多，就会减慢进度

例如：

在某Web开发项目使用实例研习会进行需求获取时，由于12位参与者对不必要的细节进行激烈的讨论，并且在每个使用实例如何工作的问题上难以达成一致意见，致使项目进展缓慢，曾一度感到灰心丧气。当把工作人员减少到6个人时，进度马上加快了，而这6个人代表了客户、系统设计者、开发者和可视化界面设计者等主要工程角色。

从极少的代表那里收集信息或者只听到呼声最高、最有舆论影响的用户的声​​音，也会造成问题

将导致忽视特定用户类的重要的需求，或者其需求不能代表绝大多数用户的需要

最好的权衡是选择用户类授权的、且也被同组用户类的其它代表所支持的产品代表者



2.7 需求获取小结

(8) 需求获取结束的标志

没有一个简单、清楚的信号暗示什么时候已完成需求获取，但是下列情形可能预示着需求获取的过程接近完成：

- 1) 用户想不出更多的使用实例（用户总是按其重要性的顺序来确定使用实例）
- 2) 用户提出新的使用实例，但可从其它使用实例的相关功能需求中获得这些
- 3) 用户开始重复原先讨论过的问题
- 4) 用户所提出的新需求比已确定的需求的优先级都低
- 5) 用户提出对将来产品的要求，而不是现在讨论的特定产品



西北工业大学
NORTHWESTERN POLYTECHNICAL UNIVERSITY

谢谢大家！
THANK YOU !

武君胜
西北工业大学 软件学院